

机器视觉技术

人工智能学院
南开大学



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

第三部分 深度学习图像处理

Deep Learning for Image Processing

➤ 讲授内容

第一部分 绪论

- 机器视觉的概念
- 程序设计基础

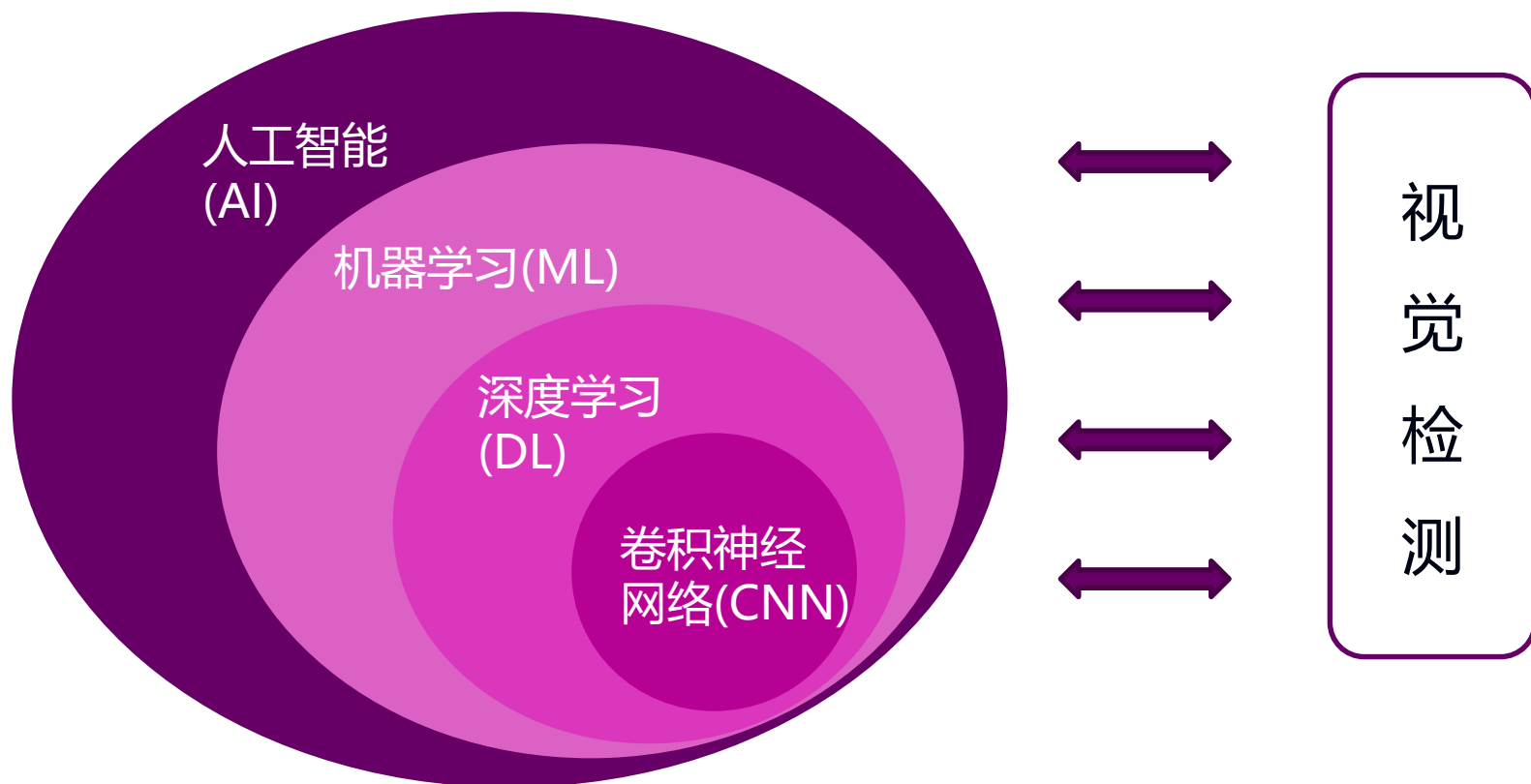
第二部分 图像处理基础

- 数字图像的概念与性质
- 图像预处理
- 二值图像处理
- 传统特征检测

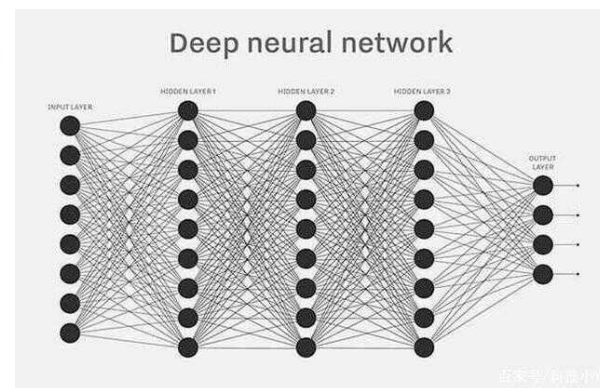
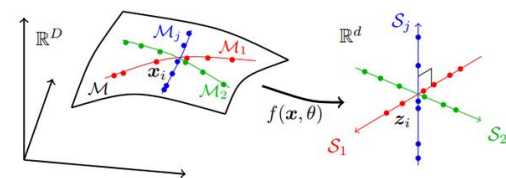
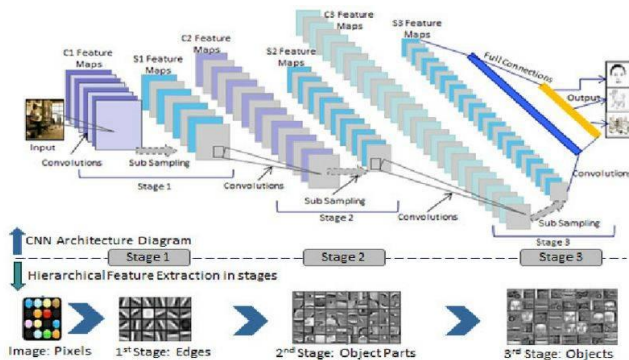
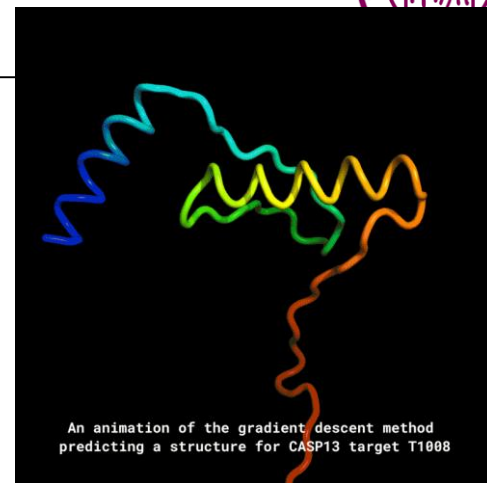
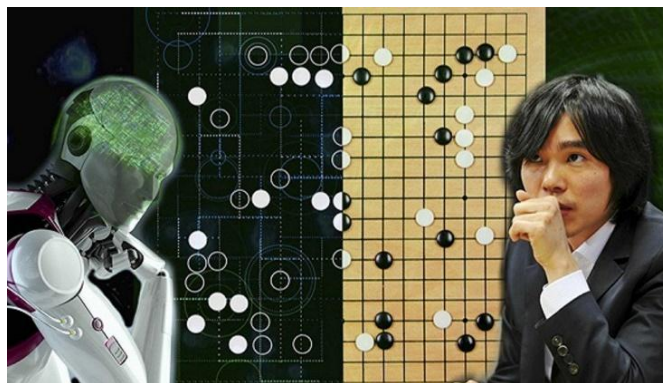
第三部分 深度学习图像处理

- 卷积神经网络
- 图像分类
- 目标检测与图像分割
- Transformer与多模态模型*

从人工智能到卷积神经网络



人工智能的发展



传统方法与卷积神经网络



IMAGENET

22,000 categories

⋮

15,000,000 images



J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li & L. Fei-Fei. CVPR, 2009.

复习：卷积运算

➤ 卷积运算

➤ 图像滤波

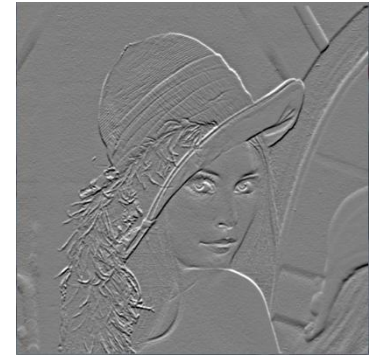
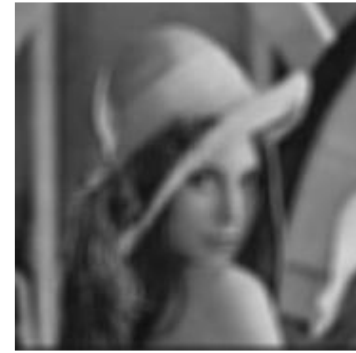
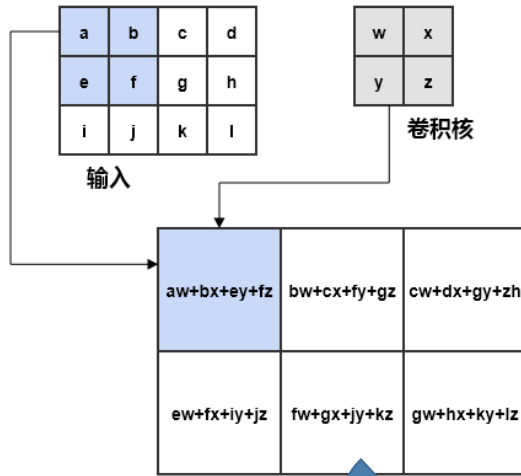
- 输出图像像素 $g(m,n)$ 的计算结果是输入图像像素 $f(m,n)$ 一个局部邻域 Ω 的亮度线性组合。邻域 Ω 中的像素贡献通过系数 h 进行加权：

$$g(m,n) = \sum_{(k,l)} \sum_{\in \Omega} f(k,l)h(m-k,n-l) = f(m,n) * h(m,n)$$

h ：滤波器、卷积掩模、核、窗口

- $\sum h = 1$ ，一般选择奇数行和奇数列的矩形邻域，容易确定邻域的中心

深度学习与卷积神经网络



用不同**卷积核**
实现不同图像处理效果

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

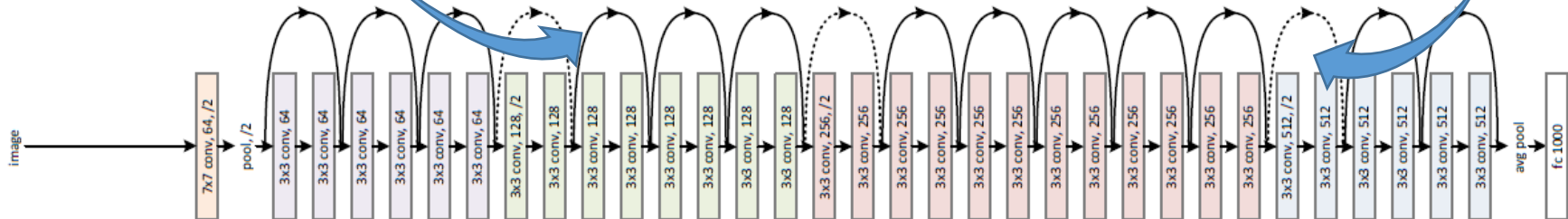
$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

卷积运算

卷积核不断堆叠形成“**深度**”网络

卷积核参数可“**学习**”

34-layer residual



传统方法与深度学习方法

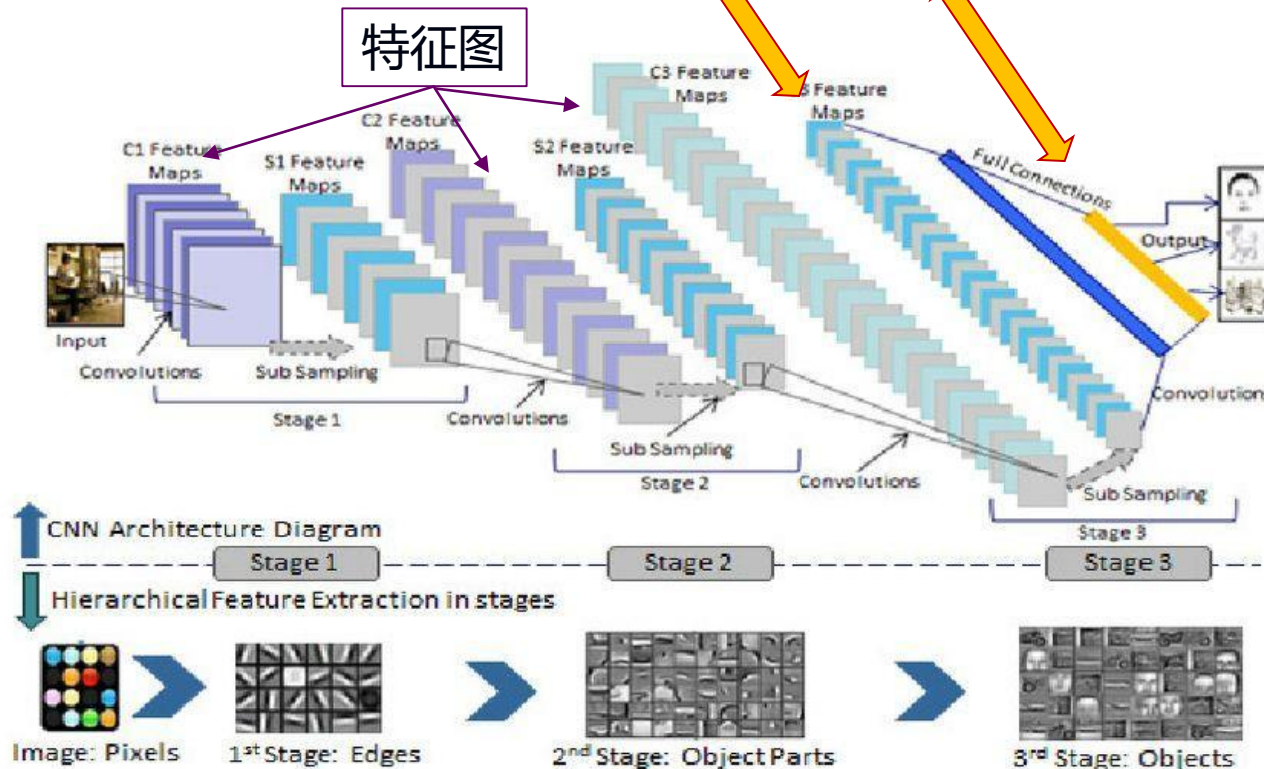


传统方法与深度学习方法



传统方法

人工设计算法
提取图像特征



卷积网络

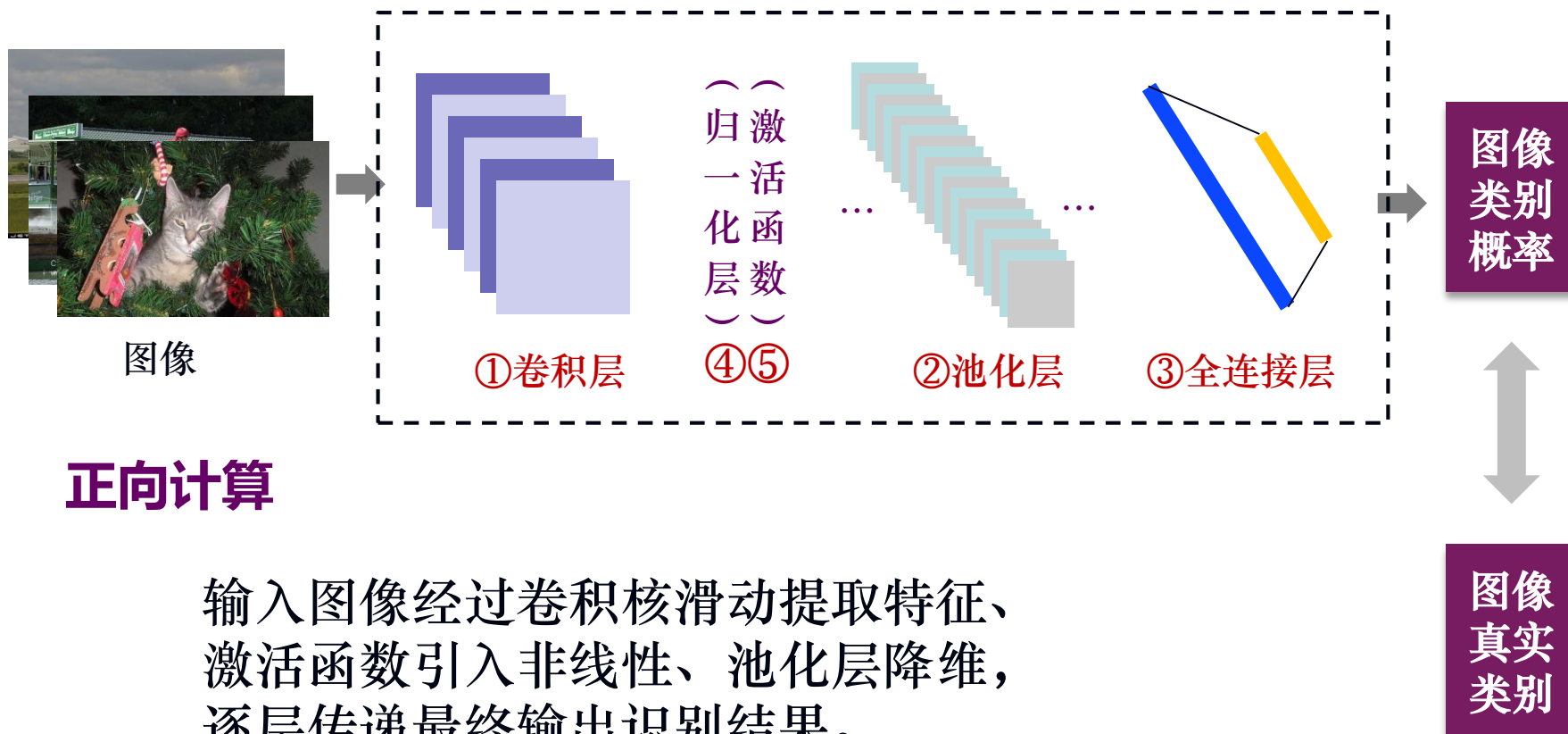
利用卷积网络
提取图像特征

主要内容



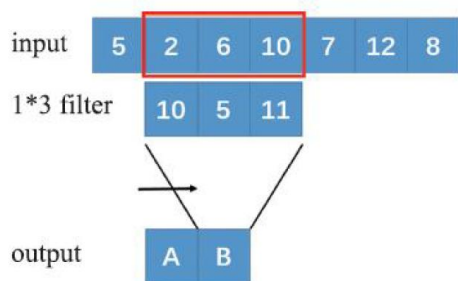
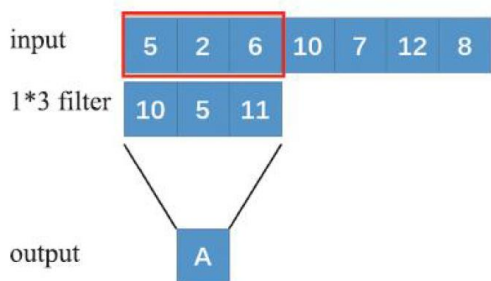
- **第六章 卷积神经网络 (第10-11周)**
 - 6.1 卷积神经网络的组成
 - 补充: Python/PyTorch编程基础*
 - 6.2 网络训练与优化
- **第七章 图像分类 (第11周)**
 - 经典图像分类网络: AlexNet, VGG, GoogLeNet, ResNet
 - 迁移学习
- **第八章 目标检测与图像分割 (第12-14周)**
- **第九章 Transformer与多模态模型 (第14周) ***

6.1 卷积神经网络的组成

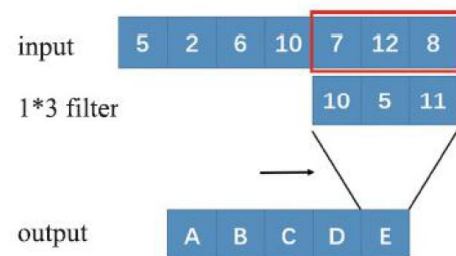


1) 卷积层

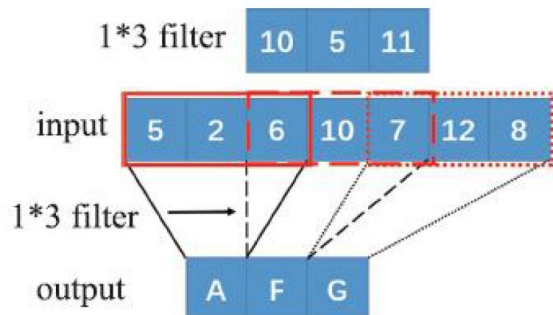
- 卷积层(Convolution): 对原图像进行卷积操作, 保留输入图像的空间特点
- 感受野: 卷积核每次滑动覆盖的格子范围



.....

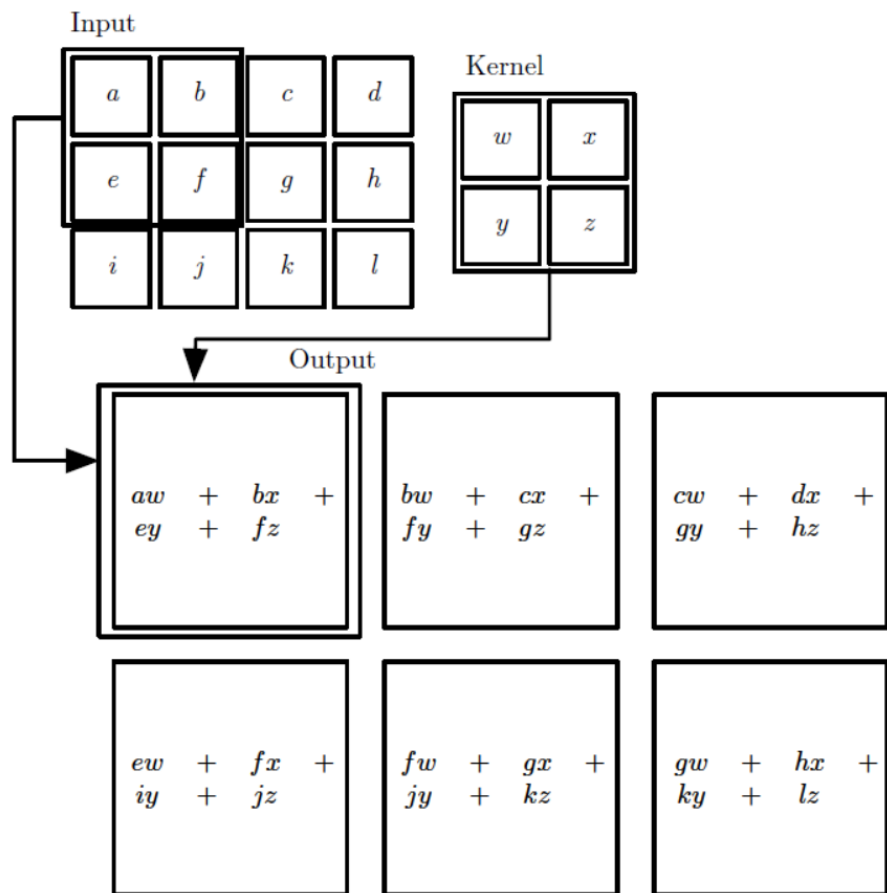


- 滑动步长



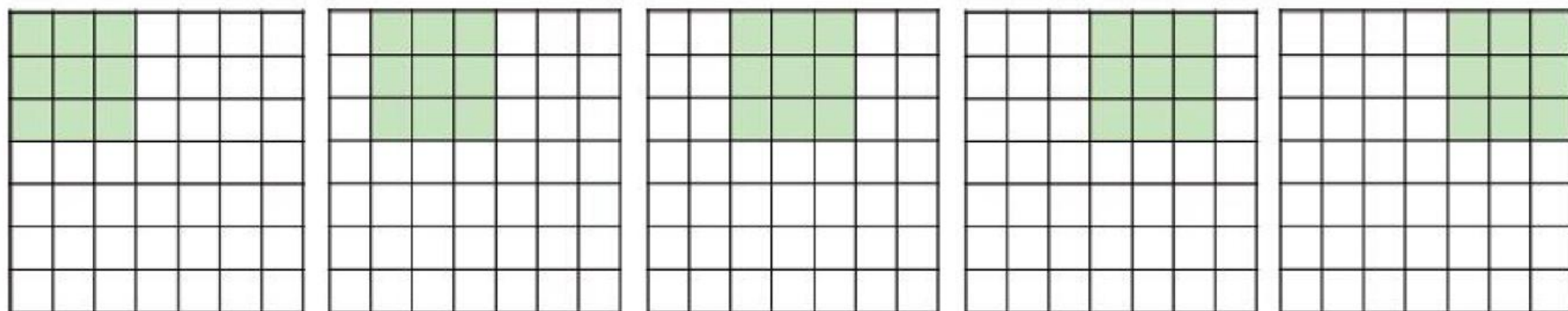
1) 卷积层

- 二维卷积: $\text{kernel}=2*2$, $\text{stride}=1$

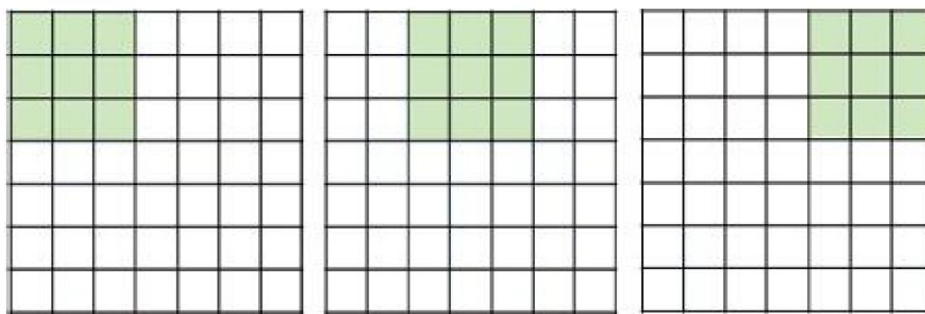


1) 卷积层

➤ 二维卷积: $\text{kernel}=3*3$



stride=1



stride=2

1) 卷积层

- 三维卷积: $\text{kernel}=5 \times 5 \times 3$, $\text{stride}=1$

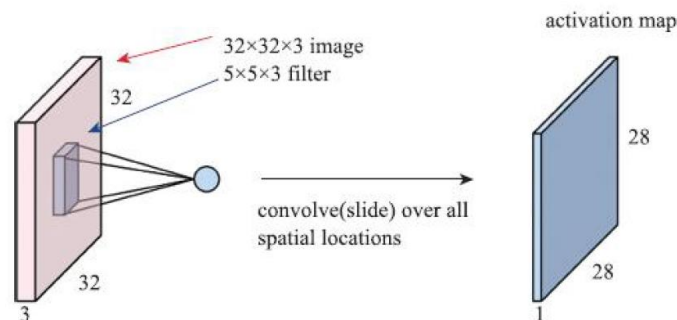
- 请思考

- 对于三维卷积核，三维卷积的结果是几维?

二维!

- 如何使三维卷积的结果还是三维?

使用多个卷积核



1) 卷积层

➤ 特征图(feature map)尺寸计算

- 输入图像尺寸: $W_1 \times H_1 \times D_1$
- 卷积核尺寸: $F \times F \times D_1$
- 卷积核个数: K
- 卷积核移动步长: S
- 填充pad大小: P

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

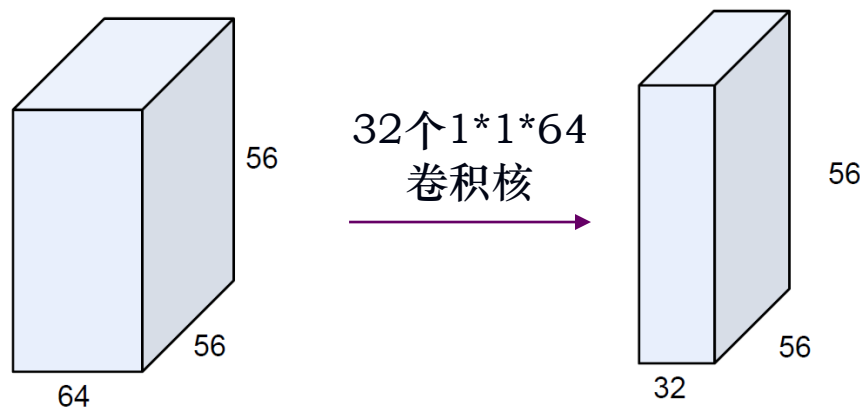
$$D_2 = K$$

特别注意!

0	0	0	0	0	0			
0								
0								
0								
0								

1) 卷积层

- 特殊的三维卷积： $\text{kernel}=1*1*n$
 - 使用 $1*1$ 的卷积核灵活地增加或减少特征图的通道数，而不改变特征图的空间尺寸
 - 降维：减少特征图通道数，显著减少后续大卷积核的参数数量和计算开销，同时保持网络的表达能力
 - 升维：将通道数变多，为后续层提供更丰富的特征表示



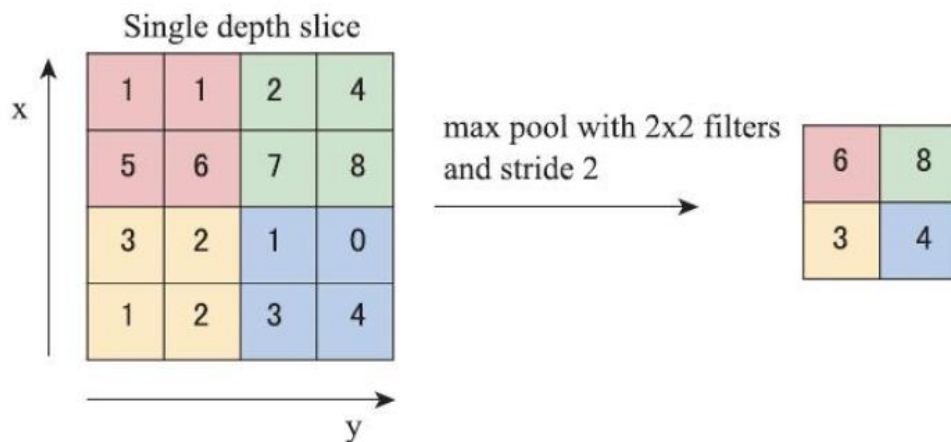
2) 池化层

- 池化层 (Pooling): 使用某一位置的相邻输出的总体统计特征来代替网络在该位置的输出
- 对图像信息进行压缩, max pooling, average pooling
 - 输入图像尺寸: $W_1 \times H_1 \times D_1$
 - 池化核尺寸: $F \times F$
 - 池化核移动步长: S
- 池化层没有需要学习的参数

$$W_2 = \frac{W_1 - F}{S} + 1$$

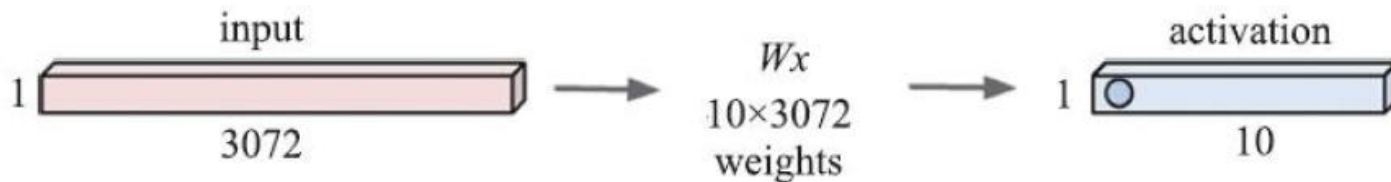
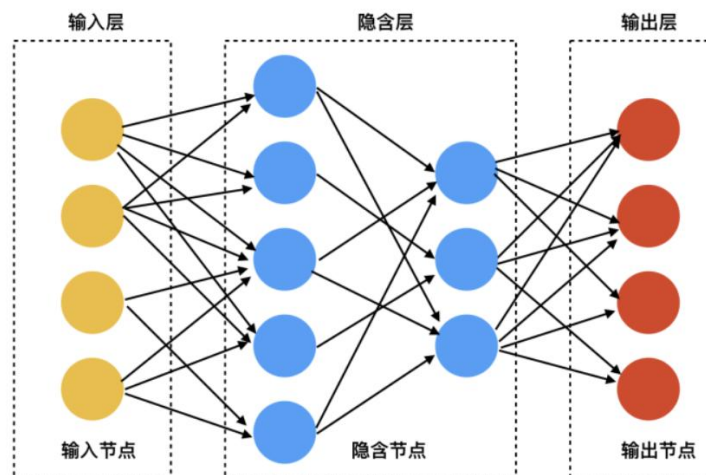
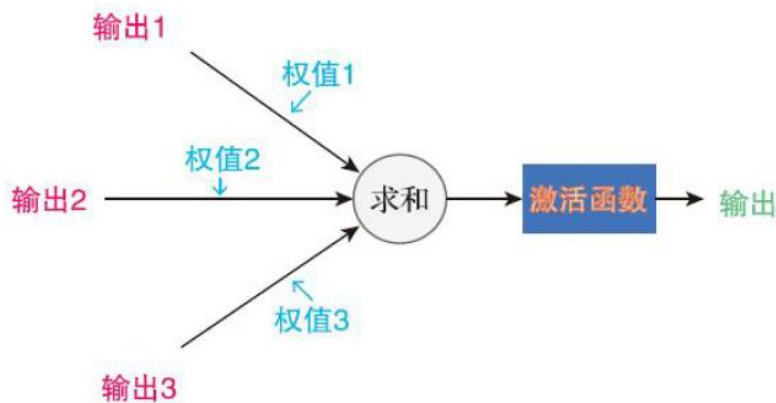
$$H_2 = \frac{H_1 - F}{S} + 1$$

$$D_2 = D_1$$



3) 全连接层

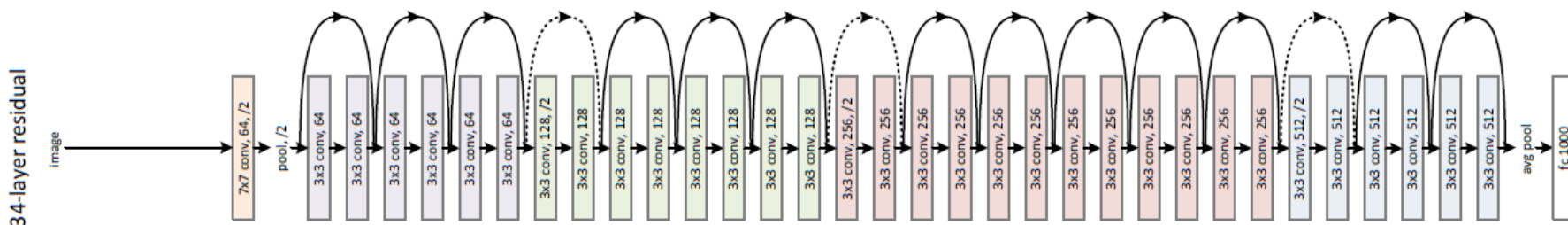
- 全连接层 (Fully Connected, FC): 类似于神经网络的隐含层, 将图像展开成1维向量作为输入
- 将向量与**权重向量(待学习参数)**做点乘, 经过激活得到结果



4) 归一化层*

➤ 层归一化(Layer Normalization)

- 神经网络的每一层的分布，会随前一层参数的更新而不断改变
- 层归一化对一个独立的样本在某一层的所有特征通道上进行归一化，使得输出具有均值为0、方差为1的分布
- 主要作用：稳定深层网络的训练，解决梯度问题



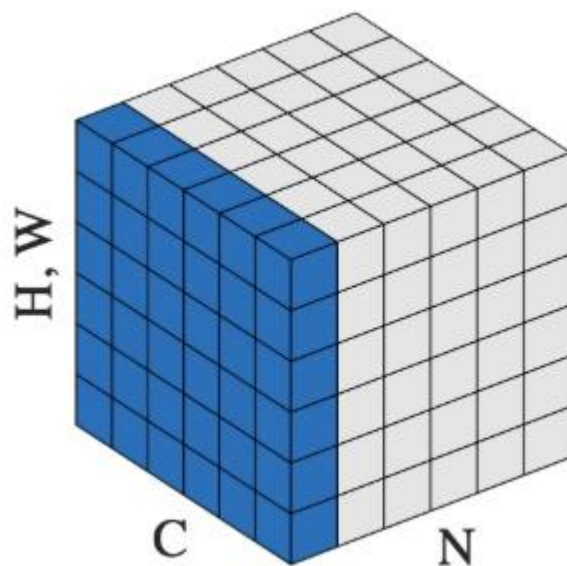
4) 归一化层*

➤ 层归一化(Layer Normalization)

- 神经网络的每一层的分布，会随前一层参数的更新而不断改变
- 层归一化对一个独立的样本在某一层的所有特征通道上进行归一化，使得输出具有均值为0、方差为1的分布
- 主要作用：稳定深层网络的训练，解决梯度问题

➤ 批归一化(Batch Normalization)

- 网络训练过程中，通常将一批数据同时送入网络，这批数据称为一个Batch
- 在一个Batch的样本上，对每一个特征通道进行归一化
- 主要作用：解决卷积神经网络中的分布漂移问题，加速神经网络收敛



4) 归一化层*

- 层归一化：对一个独立的样本在某一层的所有特征通道上进行归一化，使得输出具有均值为0、方差为1的分布
- 对于一个输入向量 $x \in R^d$ ，层归一化的计算过程如下：

- 计算均值和方差：

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

- 归一化

$$\hat{x} = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

- 缩放和平移（ γ_i, β_i 为可学习参数，归一化后，数据的分布在0点附近，通过可学习参数恢复网络的非线性表达能力）

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

4) 归一化层*

- 批归一化：在每一个小批量（Mini-Batch）数据中，对每一个通道独立进行归一化
- 假设在一个批次中有 m 个样本，对于第 k 个特征通道，批归一化的计算过程如下：

- 计算均值和方差

$$\mu_k = \frac{1}{m} \sum_{i=1}^m x_{i,k}, \sigma_k^2 = \frac{1}{m} \sum_{i=1}^m (x_{i,k} - \mu_k)^2$$

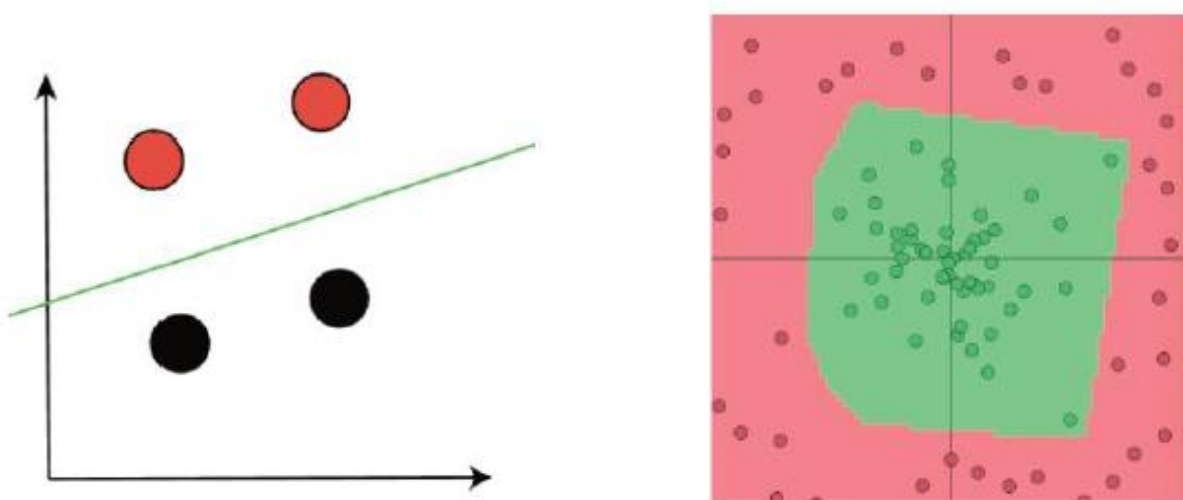
- 归一化

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_k}{\sqrt{\sigma_k^2 + \epsilon}}$$

- （同样可增加可学习参数）

5) 激活函数

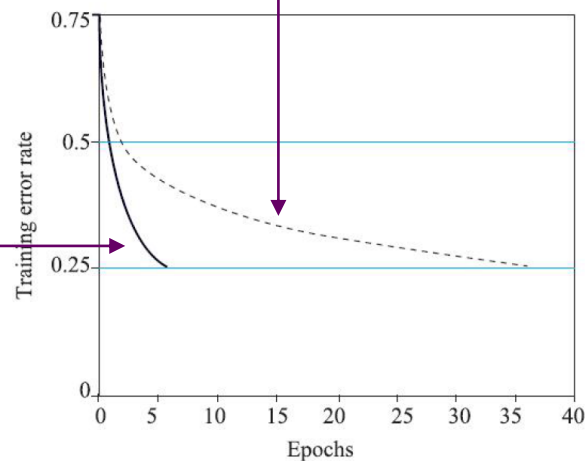
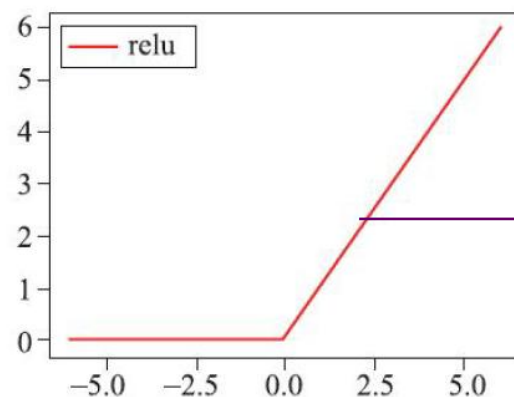
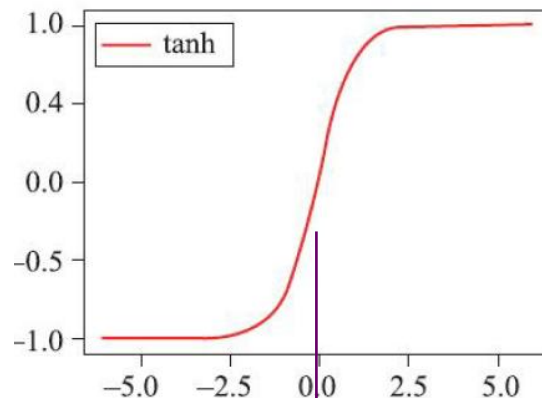
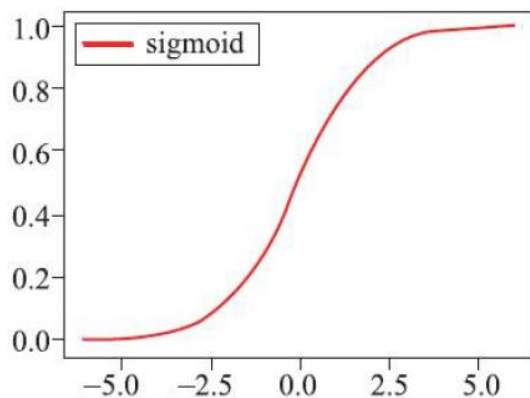
- 使用**激活函数**将输出变成输入的非线性函数，从而增强神经网络的表达能力
- 激活函数必须使用**非线性函数**



线性可分与不可分

5) 激活函数

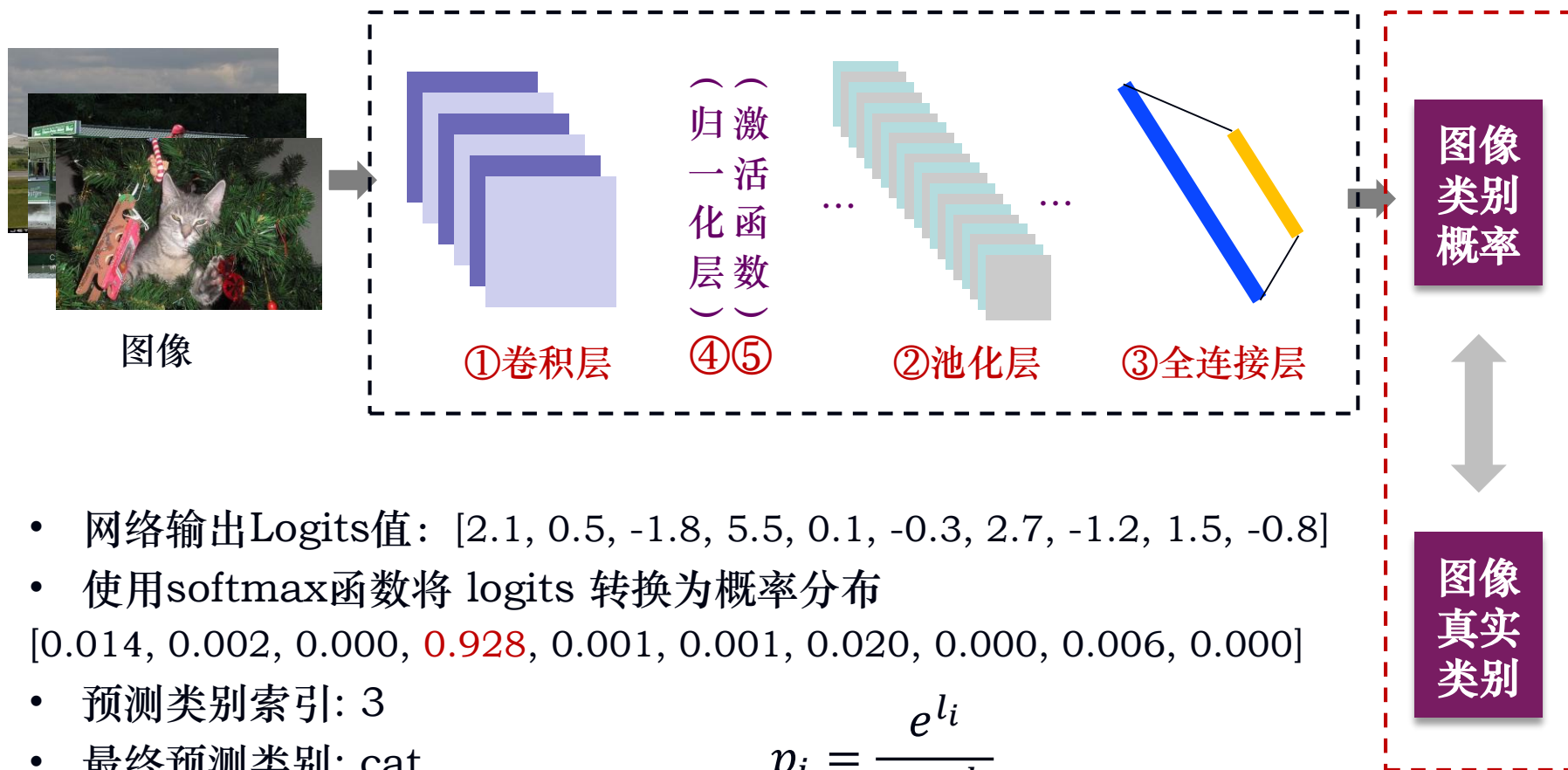
➤ 从sigmoid/tanh到Relu



5) 激活函数

Identity	Sigmoid	TanH	ArcTan
ReLU	Leaky ReLU	Randomized ReLU	Parameteric ReLU
Binary	Exponential Linear Unit	Soft Sign	Inverse Square Root Unit (ISRU)
Inverse Square Root Linear	Square Non-Linearity	Bipolar ReLU	Soft Plus

6.1 卷积神经网络的组成



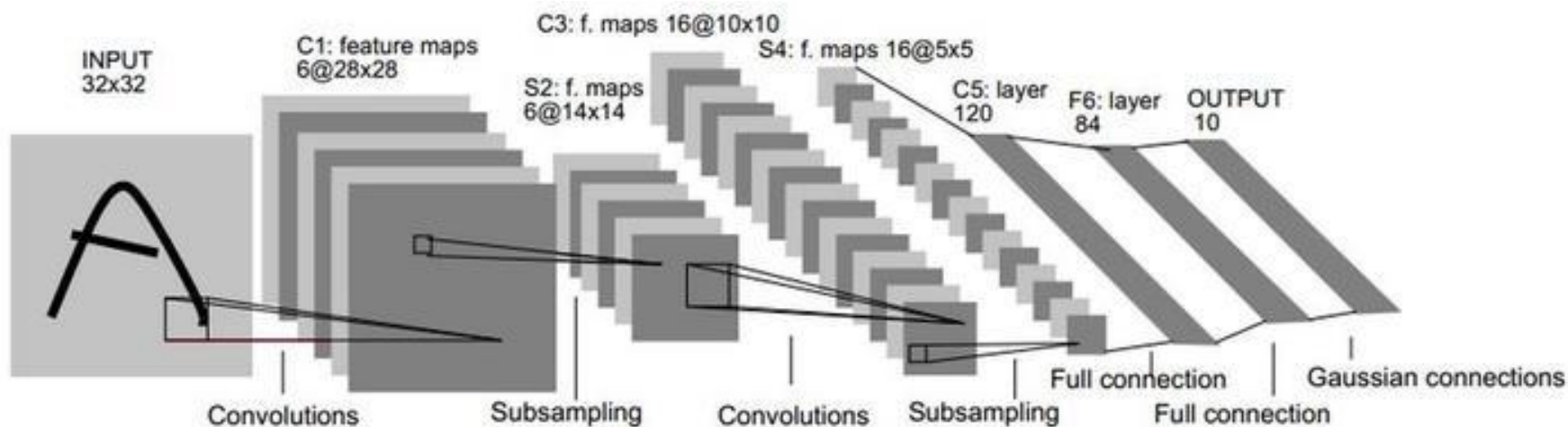
举例：LeNet-5



- 网络输入：32*32的手写数字图像
- 网络输出：识别结果（分类结果，共10个）
- **C1+S2**：卷积核(5*5)*6，无padding，池化
- C3+S4：卷积核(5*5)*16，无padding，池化
- C5：卷积核(5*5*16)*120
- F6：84个节点对应7*12编码图

input: 32*32
C1 output: 28*28*6
训练参数：
 $(5*5+1)*6=156$
S2 output: 14*14*6

偏置参数别忘了

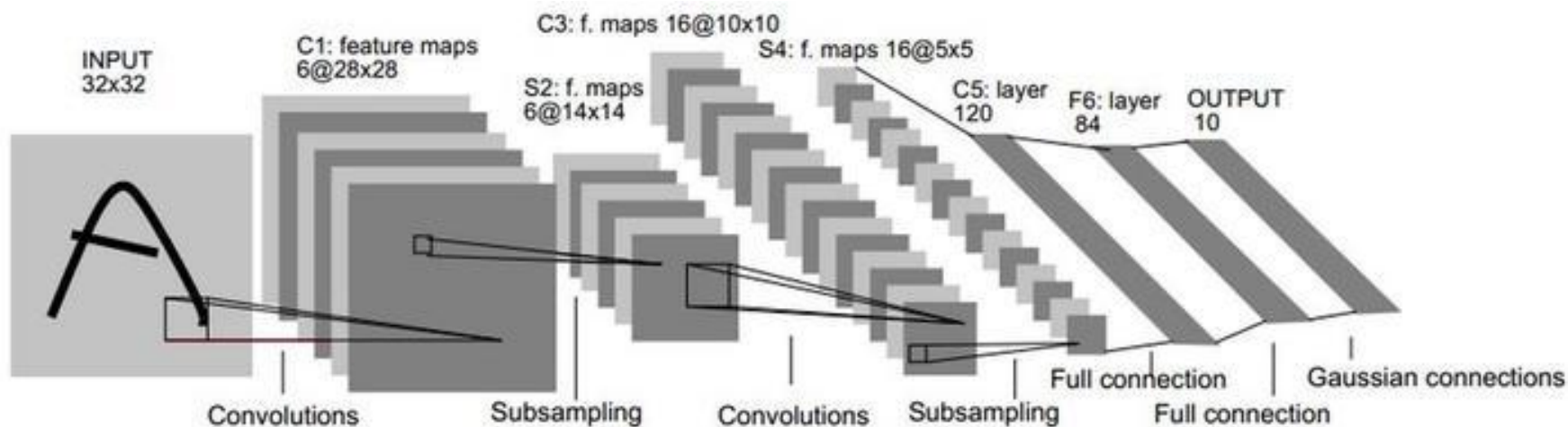


Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet-5

- 网络输入：32*32的手写数字图像
- 网络输出：识别结果（分类结果，共10个）
- C1+S2：卷积核(5*5)*6，无padding，池化
- C3+S4：卷积核(5*5)*16，无padding，池化
- C5：卷积核(5*5*16)*120
- F6：84个节点对应7*12编码图

S2 output: $14 \times 14 \times 6$
 C3 output: $10 \times 10 \times 16$
 训练参数：
 $(5 \times 5 \times 6 + 1) \times 16 = 2416$



Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet-5

- 网络输入：32*32的手写数字图像
- 网络输出：识别结果（分类结果，共10个）
- C1+S2：卷积核(5*5)*6，无padding，池化
- C3+S4：卷积核(5*5)*16，无padding，池化
- **C5**：卷积核(5*5*16)*120
- **F6**：84个节点对应7*12编码图

S4 output: $5*5*16$

C5 output: 120

训练参数:

$$(5*5*16+1)*120=48120$$

F6 output: 84

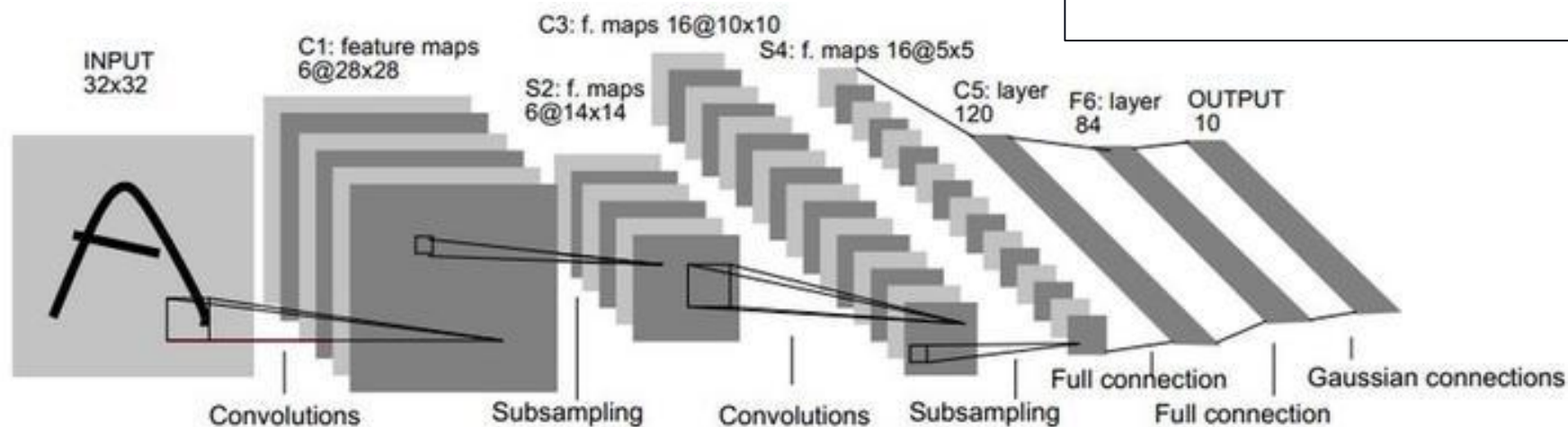
训练参数:

$$(120+1)*84=10164$$

OUTPUT: 10

训练参数:

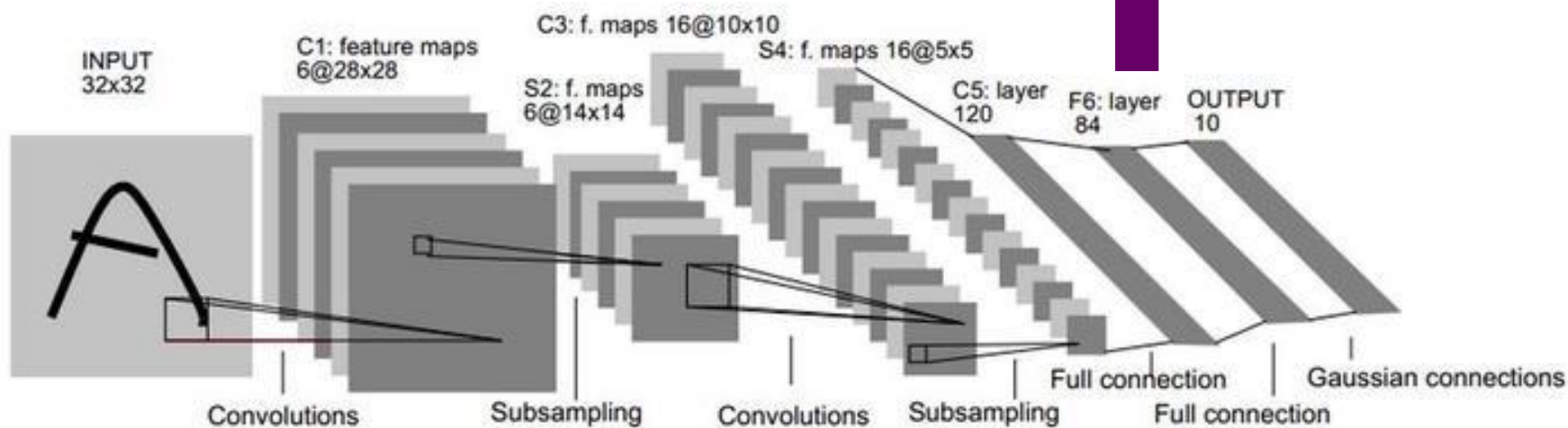
$$(84+1)*10=850$$



Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet-5

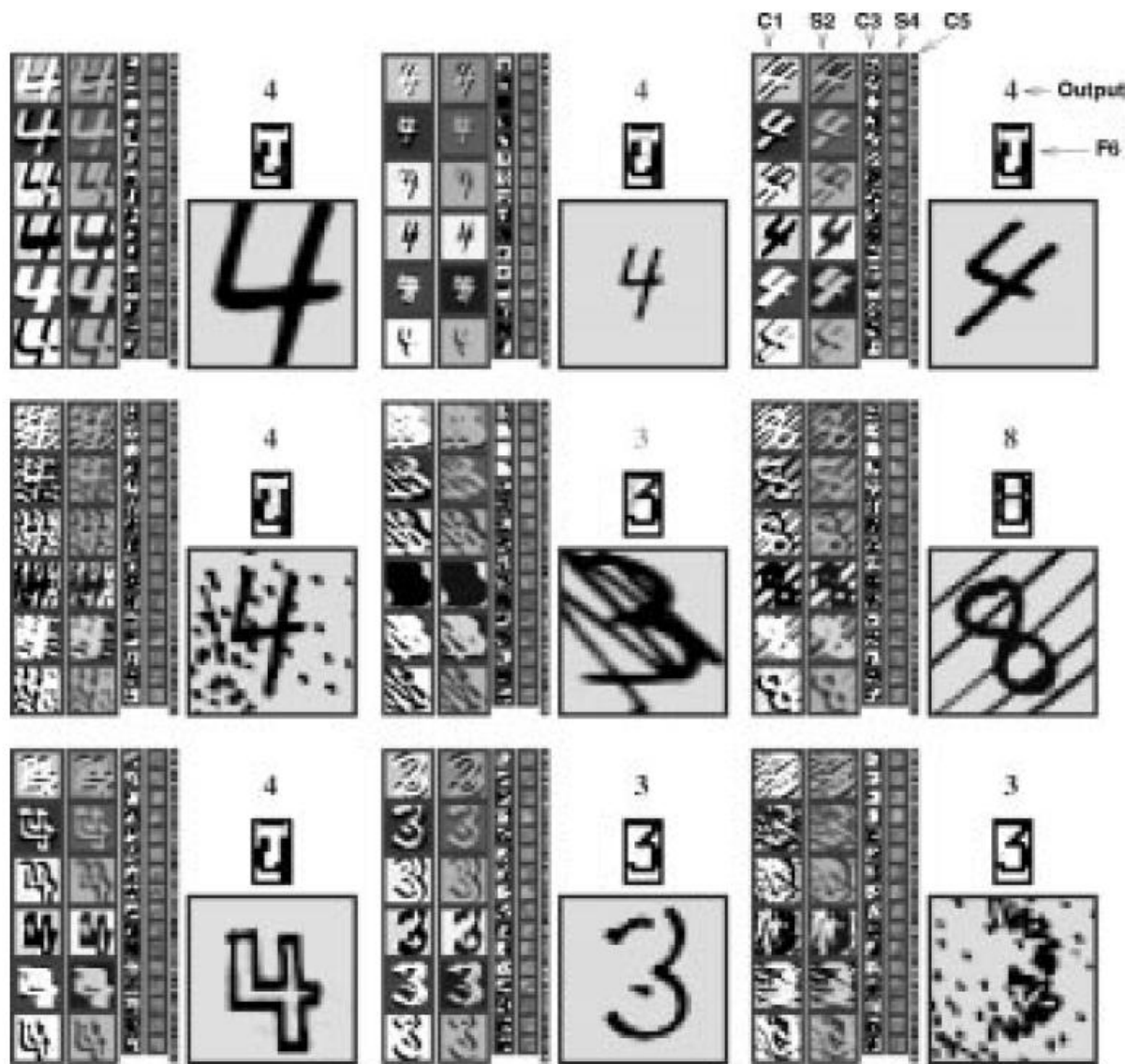
- 网络输入：32*32的手写数字图像
- 网络输出：识别结果
- C1+S2：卷积核(5*5)*6，池化
- C3+S4：卷积核(5*5)*16，池化
- C5：卷积核(5*5*16)*120
- F6：84个节点对应7*12编码图



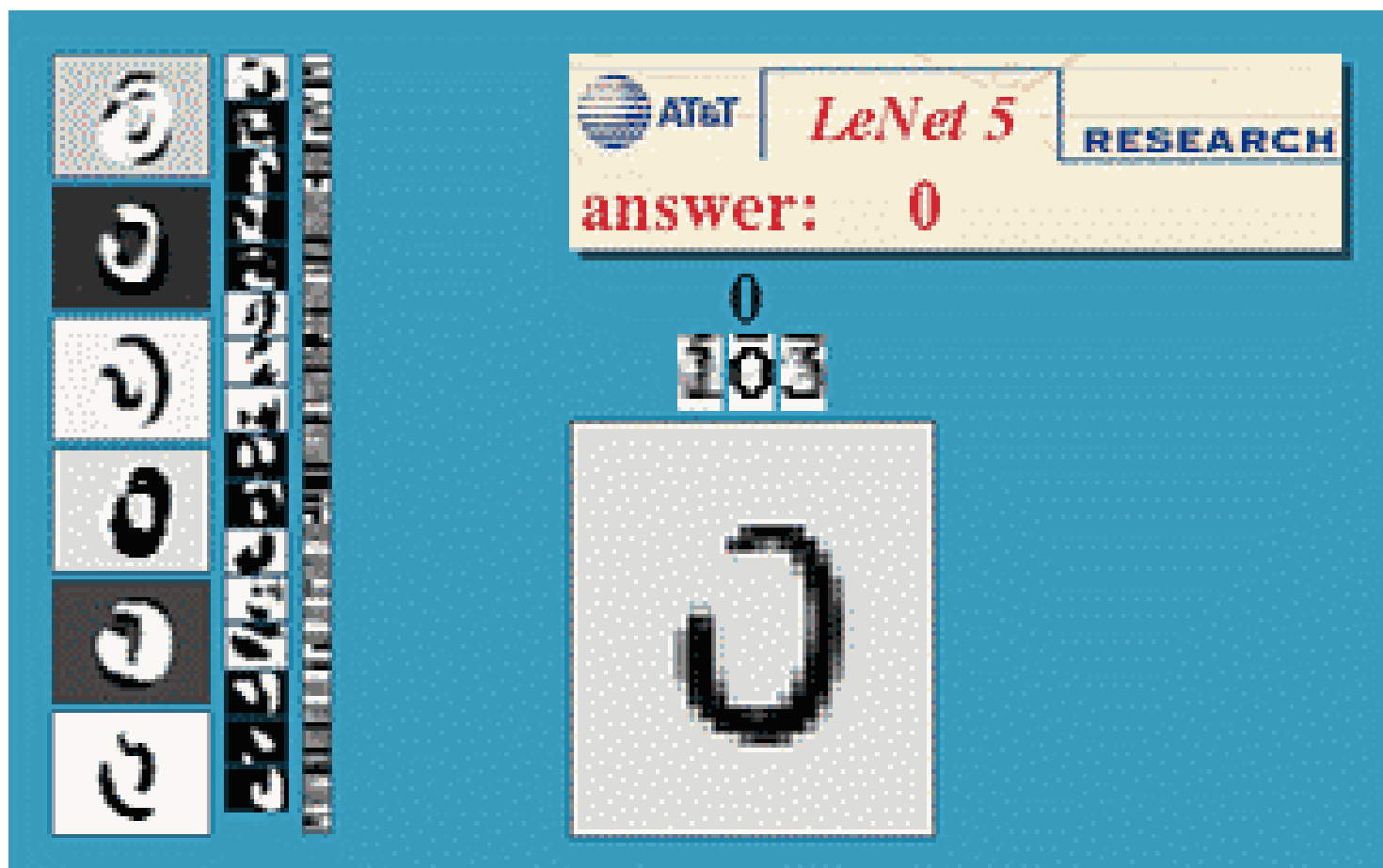
Gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]

举例：LeNet

MNIST, 包含10个类别, 6万张 28×28 图像



举例：LeNet-5



补充1: Python简介

- Python 是一种广泛使用的高级、通用编程语言，以其简洁、易读的语法和强大的功能而著称，在1991年首次发布。
- Python语言核心特点：
 - 简洁易读：Python的语法接近自然语言，并强制使用缩进来表示代码块，这使得代码结构清晰，易于理解和维护。
 - 开源免费：Python是开源软件，任何人都可以免费使用、学习和分发，这促进了其庞大的社区和生态系统的形成。
 - 跨平台：Python是解释型语言，可以在Windows、macOS、Linux等多种操作系统上运行，无需修改代码。
 - 丰富的库：Python其标准库和第三方库（如NumPy, Pandas, Django等）极其丰富，覆盖了从Web开发到人工智能的几乎所有领域，极大地提高了开发效率。
 - 面向对象：Python全面支持面向对象编程，同时也支持面向过程等多种编程范式，非常灵活。

1) Python基本语法：与C++对比

➤ 变量声明：无需关注数据类型

C++	Python
<code>int a = 5;</code>	<code>a = 5</code>
<code>string name = "Alice";</code> // 需要引入string库	<code>s = "hello"</code>
// 数组 <code>int numbers[] = {1, 2, 3};</code>	# 列表，支持多种数据类型 <code>mixed_list = [1, 2, "three", True]</code>

➤ 循环

C++	Python
// 基于计数器 <code>for (int i = 0; i < 5; i++)</code> <code>{</code> <code>cout << i << endl;</code> <code>}</code>	# 直接遍历序列中的每一个元素 # 使用缩进划分代码块，无需{} # 不要忘记最后的冒号: <code>for i in range(5):</code> <code>print(i)</code>

1) Python基本语法：与C++对比

➤ 函数定义

C++	Python
<pre>// 明确返回类型，更加严谨 int add(int a, int b) { return a + b; }</pre>	<pre># 使用def 定义函数，不需要指定参数 类型和返回值类型，更加简洁 # 不要忘记最后的冒号: def add(a, b): return a + b</pre>

➤ 类定义

- 构造函数：C++ 使用与类同名函数；Python 使用 `__init__` 函数
- 实例本身：C++使用this 指针隐式使用，Python 的类方法必须显式包含self参数代表实例本身
- 成员变量：C++ 必须在类中声明成员变量；Python 可以在 `__init__` 中动态添加

2) Python内置数据结构

1. 列表 (List): Python中最常用的数据结构, 相当于C++的 `std::vector`, 但可存储不同类型的对象, 功能更强大
 - 定义: 使用方括号 `[]`
 - 特性: 动态扩容、类型混合、切片 (Slicing)

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")           # 添加元素
print(fruits[1])                # 索引访问: banana
print(fruits[-1])               # 负数索引表示倒数第一个: date
print(fruits[1:3])              # 切片: ['banana', 'cherry'] (左闭右开)
```

列表：切片 (Slicing)

- 切片：快速、优雅地从序列（如列表、元组）中提取一部分元素
- 语法结构： `sequence[start:stop:step]` （左闭右开）
 - start: 起始索引，默认值为0，包含该位置的元素
 - stop: 结束索引，默认值为序列长度，**不包含该位置的元素**
 - step: 步长，默认值为1，可跳跃间隔，**可以是负数**
 - **切片支持负数索引**，-1代表最后一个元素，-2 代表倒数第二个，以此类推

- **举例** `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

<code>nums[2:5]</code>	<code>[2, 3, 4]</code>
<code>nums[:4]</code>	<code>[0, 1, 2, 3]</code>
<code>nums[7:]</code>	<code>[7, 8, 9]</code>
<code>nums[::2]</code>	<code>[0, 2, 4, 6, 8]</code>
<code>nums[1::3]</code>	<code>[1, 4, 7]</code>

列表：切片 (Slicing)

- 切片：快速、优雅地从序列（如列表、元组）中提取一部分元素
- 语法结构： sequence[start:stop:step] (左闭右开)
 - start: 起始索引，默认值为0，包含该位置的元素
 - stop: 结束索引，默认值为序列长度，**不包含该位置的元素**
 - step: 步长，默认值为1，可跳跃间隔，**可以是负数**
 - **切片支持负数索引**，-1代表最后一个元素，-2 代表倒数第二个，以此类推
- 举例 `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

<code>nums[-3:]</code>	<code>[7, 8, 9]</code>
<code>nums[:-3]</code>	<code>[0, 1, 2, 3, 4, 5, 6]</code>
<code>nums[::-1]</code>	<code>[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]</code>
<code>nums[7:2:-1]</code>	<code>[7, 6, 5, 4, 3]</code>

列表：推导式（Comprehension）

- 列表推导式：在一行代码中创建列表的简洁方法
- 语法结构
 - 基本公式：[表达式 for 变量 in 可迭代对象]
 - 带条件的公式：[表达式 for 变量 in 可迭代对象 if 条件]
- 举例
 - 生成从0到9的平方的列表

```
[x**2 for x in range(10)]
```
 - 生成从0到9中偶数平方的列表

```
[x**2 for x in range(10) if x % 2 == 0]
```
 - 多重循环推导式：生成连续的二维坐标点

```
[(x, y) for x in range(3) for y in range(3)]
```


2) Python内置数据结构

2. 元组 (Tuple): 只读列表, 一旦创建, 就不能修改。通常用于存储“结构化的一条数据”

- 定义: 使用圆括号 ()
- 特性: 不可变性, 类似于 C++ 中的 const 数组, 保证了数据安全, 且内存占用比列表更小

```
# 定义元组
point = (10, 20)
# 解包, 把元组里的值赋给多个变量
x, y = point
print(x, y) # 10 20

# point[0] = 5 # 报错! 元组不可修改
```

2) Python内置数据结构

3. 字典 (Dict): 基于哈希表实现, 通过“键”来查找“值”, 查找速度极快 ($O(1)$ 复杂度)

- 定义: 使用花括号 {}, 格式为 {键: 值}
- 特性: 键必须唯一且可哈希, 通常用字符串或数字做键

```
person = {  
    "name": "Alice",  
    "age": 25  
}  
  
print(person["name"]) # 直接访问: Alice  
print(person["job"])  # 如果键不存在, 会报 KeyError  
print(person.get("job")) # 安全访问: None
```

2) Python内置数据结构

4. 集合 (Set): 数学集合, 保证元素不重复, 可进行数学上的交集、并集运算

- 定义: 使用花括号 {}, 空集合必须用 set()
- 特性: 自动去重, 添加重复元素会被自动忽略; 集合里的元素没有顺序, 不能通过索引访问

```
nums = {1, 2, 3, 3, 4}
print(nums)  # {1, 2, 3, 4} -> 3 被自动去重了

nums.add(5)      # 增加元素
nums.discard(2)  # 删除元素, 如果不存在也不会报错
```

3) 多维数组

- NumPy (Numerical Python) 是 Python科学计算的核心库，提供高性能的**多维数组对象**和丰富的数学函数
- 提供了名为 ndarray (N-dimensional array) 的对象，专门用于存储和处理多维数组。底层采用C语言编写，**运算速度极快**，且支持向量化运算
- 创建多维数组

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]]) # 从列表转换
zeros = np.zeros((3, 4)) # 3行4列的全0数组
ones = np.ones((2, 2)) # 2行2列的全1数组
range_arr = np.arange(12).reshape(3, 4) # 生成0到11的数字，并重塑为 3行4列
range_arr2 = np.arange(0, 20, 2).reshape(2, 5) # 生成0到18的偶数，并重塑为 2行5列
```

3) 多维数组

➤ ndarray的核心属性:

- ndim: 维度的数量
- shape: 数组的形状
- size: 元素总个数
- dtype: 数据类型

```
arr = np.array([[1, 2, 3], [4, 5, 6]]) # 从列表转换
zeros = np.zeros((3, 4)) # 3行4列的全0数组
ones = np.ones((2, 2)) # 2行2列的全1数组
range_arr = np.arange(12).reshape(3, 4) # 生成0到11的数字, 并重塑为 3行4列

print(arr.ndim) # 2
print(zeros.shape) # (3, 4)
print(ones.size) # 4
print(range_arr.dtype) # int32
```

3) 多维数组：索引与切片

1. 基础切片：方括号与冒号

- 语法和列表类似：start:stop:step（左闭右开）
- 在多维数组中，使用逗号来分隔不同维度
- 举例

```
range_arr = np.arange(12).reshape(3, 4)
```

- | | |
|-----------------------|------------------------------------|
| ➤ 取数组第一行 | <code>range_arr[0, :]</code> |
| ➤ 取数组第二列 | <code>range_arr[:, 1]</code> |
| ➤ 取数组的第1/2行、2/3列子矩阵 | <code>range_arr[0:2, 1:3]</code> |
| ➤ 取数组的第1/2行反序、2/3列子矩阵 | <code>range_arr[1::-1, 1:3]</code> |

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```


3) 多维数组：索引与切片

1. 基础切片：方括号与冒号

- 语法和列表类似：start:stop:step（左闭右开）
- 在多维数组中，使用逗号来分隔不同维度

2. 花式索引：在多维数组中取特定元素

- 举例：取 (0, 1) 和 (2, 3) 位置的元素

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
rows = np.array([0, 2])
cols = np.array([1, 3])
print(range_arr[rows, cols])
```

3. 布尔索引：条件筛选，用一个布尔数组过滤数据

```
print(range_arr[range_arr > 5]) # 筛选出所有大于5的元素
# 筛选出大于3且小于9的元素
print(range_arr[(range_arr > 3) & (range_arr < 9)])
```

3) 多维数组：广播机制

- 广播机制允许形状不同的数组进行算术运算，而无需手动去复制数据来补齐维度
- 广播机制的三大法则：决定两个数组是否可以广播
 - 法则 1：维度对齐从右开始
 - 法则 2：维度相等或其中一个为1，可进行广播
 - (2, 3) 和 (2, 3) # 维度相等 → 兼容
 - (2, 3) 和 (1, 3) # 第1维: 2 vs 1 → 兼容 (1可扩展为2)
 - (3, 4, 5) 和 (4, 5) # 从右对齐: 4=4, 5=5 → 兼容
 - (5, 1, 4) 和 (5, 3, 4) # 第2维: 1 扩展为 3 → 兼容
 - 法则 3：缺失维度自动补 1，如果数组维度不同，较小数组的左边自动补 1
 - (2, 3, 4) 和 (4,) 从右对齐+左边补1 → (2, 3, 4)和(1, 1, 4)

补充2: PyTorch基础

➤ 深度学习框架

➤ PyTorch vs. TensorFlow vs. MindSpore

Caffe
(UC Berkeley)



Caffe2
(Facebook)
mostly features absorbed
by PyTorch

Torch
(NYU / Facebook)



PyTorch
(Facebook)

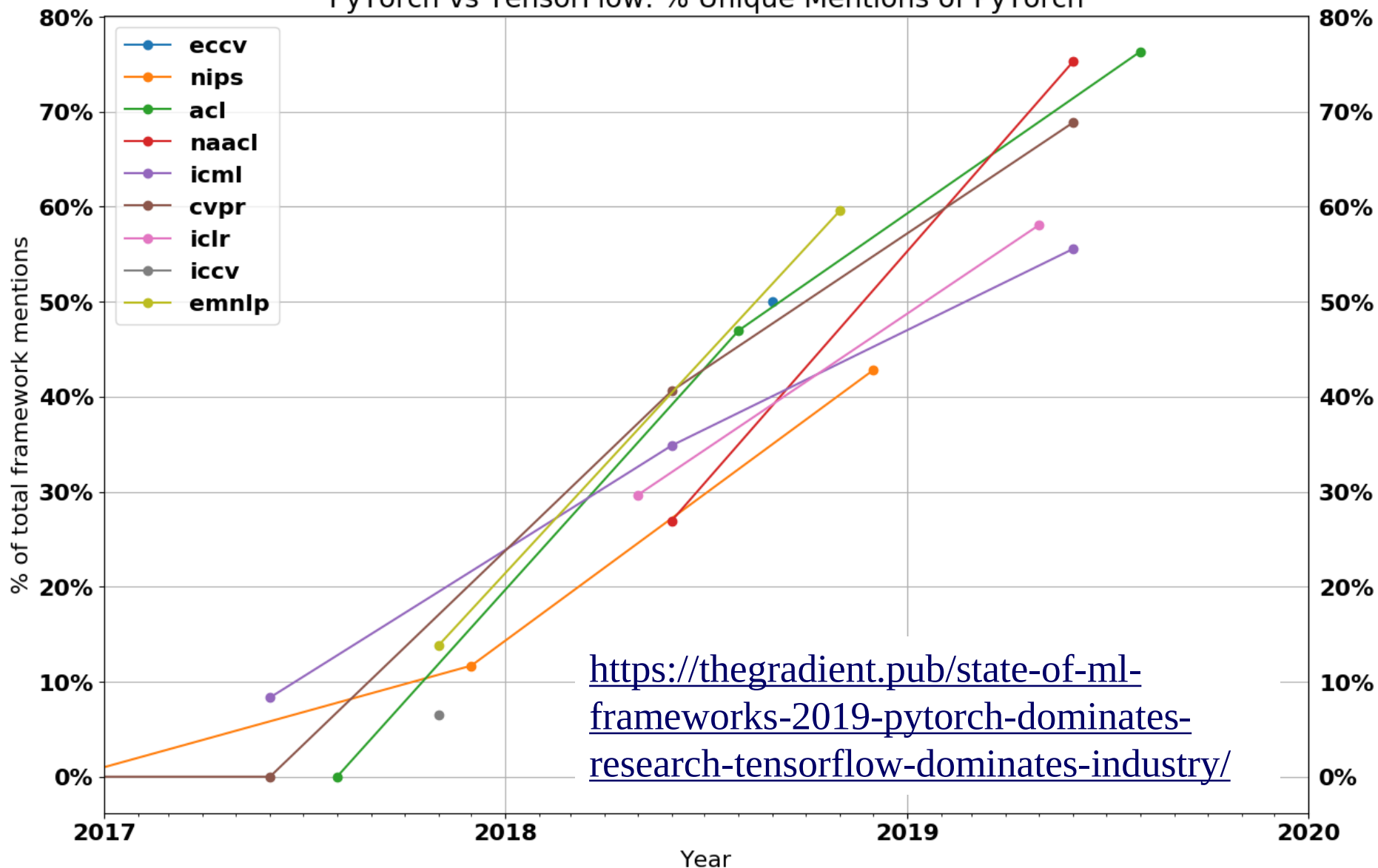
Theano
(U Montreal)



TensorFlow
(Google)



PyTorch vs TensorFlow: % Unique Mentions of PyTorch



深度学习框架



➤ PyTorch(PT) vs. TensorFlow(TF)

	PT	PT	PT	TF	TF	TF
CONFERENCE	2018	2019	GROWTH	2018	2019	GROWTH
CVPR	82	280	240%	116	125	7.7%
NAACL	12	66	450%	34	21	-38.2%
ACL	26	103	296%	34	33	-2.9%
ICLR	24	70	192%	54	53	-1.9%
ICML	23	69	200%	40	53	32.5%

<https://thegradient.pub/state-of-ml-frameworks-2019-pytorch-dominates-research-tensorflow-dominates-industry/>

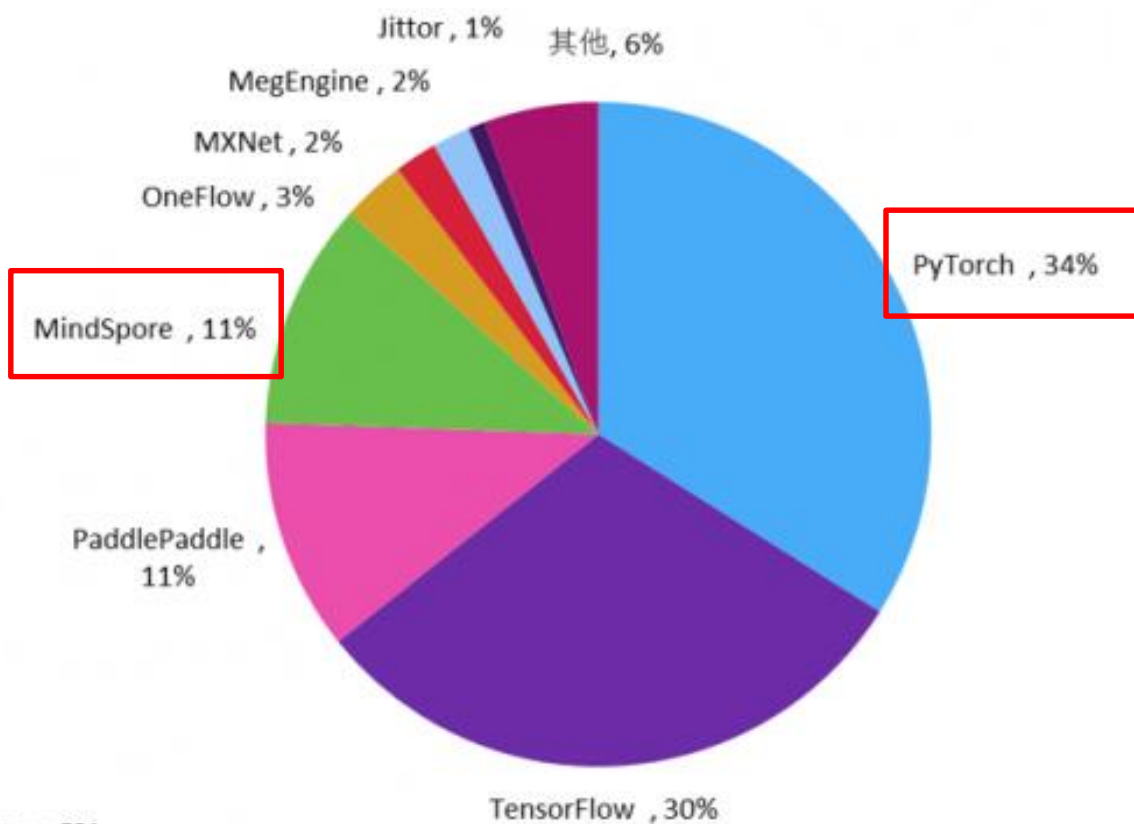
深度学习框架



➤ 行业研究机构 Omdia调研



Q11 中国开发者主流人工智能框架使用率排名

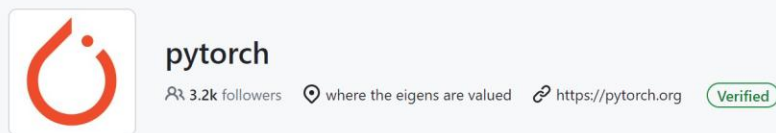


Note: n=531
Source: Omdia

© 2022 Omdia

补充2: PyTorch基础

- <https://pytorch.org/>
- <https://github.com/pytorch/pytorch>



Overview Repositories 68 Projects 11 Packages People 47

Pinned

 **pytorch** Public
Tensors and Dynamic neural networks in Python with strong GPU acceleration
C++ 65.4k 18k

 **examples** Public
A set of examples around pytorch in Vision, Text, Reinforcement Learning, etc.
Python 20.1k 9.2k

 **tutorials** Public
PyTorch tutorials.
Python 6.9k 3.7k

 **vision** Public
Datasets, Transforms and Models specific to Computer Vision
Python 13.7k 6.6k

 **text** Public
Models, data loaders and abstractions for language processing, powered by PyTorch
Python 3.3k 800

 **extension-cpp** Public
C++ extensions in PyTorch
Python 834 181

PyTorch Recipes [+]

Introduction to PyTorch [-]

Learn the Basics

Quickstart

Tensors

Datasets & DataLoaders

Transforms

Build the Neural Network

Automatic Differentiation with `torch.autograd`

Optimizing Model Parameters

Save and Load the Model

Introduction to PyTorch on YouTube [+]

Learning PyTorch [+]

Image and Video [-]

TorchVision Object Detection Finetuning Tutorial

Transfer Learning for Computer Vision Tutorial

Adversarial Example Generation

DCGAN Tutorial

Spatial Transformer Networks Tutorial

Optimizing Vision Transformer Model for Deployment



补充2: PyTorch基础

➤ Tensor (张量)

- 深度学习框架中最基本的数据结构，可理解为多维数组
- 从数学角度看，Tensor是广义的矩阵
- Tensor在很多方面与NumPy的ndarray非常相似，可以被看作是GPU 加速版且支持自动求导的NumPy数组

特性	NumPy (ndarray)	PyTorch (Tensor)
硬件加速	仅支持 CPU	支持 CPU 和 GPU
自动求导	不支持	支持 (Autograd)
主要用途	通用科学计算、数据分析	深度学习模型开发与训练

补充2: PyTorch基础

➤ Tensor的核心属性

- shape (形状)、dtype (数据类型, torch.float32/float16/int64...)、device (设备, 是 'cpu' 还是 'cuda:0')

➤ 创建 Tensor

- 从Python列表创建: torch.tensor([[1, 2], [3, 4]])
- 使用torch.arange(start, end, step)

➤ PyTorch Tensor 和 NumPy 数组的互换

- NumPy → PyTorch: 使用 torch.from_numpy(ndarray)
- PyTorch → NumPy: 使用 tensor.numpy()

```
np_array = np.array([1, 2, 3, 4, 5])  
tensor = torch.from_numpy(np_array)  
tensor = torch.tensor([1, 2, 3, 4, 5])  
np_array = tensor.numpy()
```

补充2: PyTorch基础

- **索引与切片**：和NumPy几乎一样，支持逗号分隔的多维索引
- **维度变换**
 - `view()` / `reshape()`：改变形状，区别在于对内存连续性的要求不同
 - `unsqueeze(dim)`：增加维度
 - `squeeze(dim)`：减小维度
 - `permute(*dims)`：维度置换，例如将 (C, H, W) 变为 (H, W, C)

```
x = torch.arange(12)
a = x.view(2, 2, 3) # 2x2x3 三维张量
x_reshape = x.reshape(3, 4) # 3x4 二维张量
a1 = a.unsqueeze(0) # 1x2x2x3, 在第0维增加
a2 = a1.squeeze(0) # 2x2x3, 删除第0维
a3 = a.permute(2, 0, 1) # 把第3维调到最前
```

PyTorch: 卷积层

Conv2d

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1,  
groups=1, bias=True, padding_mode='zeros', device=None, dtype=None) [SOURCE]
```

➤ 特征图(feature map)尺寸计算

- 输入图像尺寸: $W_1 \times H_1 \times D_1$
- 卷积核尺寸: $F \times F \times D_1$
- 卷积核个数: K
- 卷积核移动步长: S
- 填充pad大小: P

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

PyTorch: 卷积层

Conv2d

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

```
CLASS torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None) [SOURCE]
```

- `in_channels`: 输入数据的通道数
- `out_channels`: 输出特征图的通道数，也是卷积核的数量
- `kernel_size`: 卷积核的尺寸
- `Stride`: 卷积核滑动的步长，默认是 1
- `Padding`: 在输入边缘填充的像素数，默认是 0
- `Dilation`: 空洞卷积的膨胀率，默认为 1
- `Groups`: 分组卷积的组数，默认为 1，设为 `in_channels` 时，就是深度可分离卷积（Depthwise Conv），用于减少计算量。
- `Bias`: 是否添加偏置项，默认为 `True`；如果后面接了 `BatchNorm`，通常设为 `False`

PyTorch: 池化层

MaxPool2d

<https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html#maxpool2d>

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False) [SOURCE]
```

- 池化层 (Pooling): 使用某一位置的相邻输出的总体统计特征来代替网络在该位置的输出
- 对图像信息进行压缩, **max pooling**
 - 输入图像尺寸: $W_1 \times H_1 \times D_1$
 - 池化核尺寸: $F \times F$
 - 池化核移动步长: S

PyTorch: 池化层

MaxPool2d

<https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html#maxpool2d>

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False,  
                        ceil_mode=False) [SOURCE]
```

- `kernel_size`: 池化窗口的尺寸
- `Stride`: 窗口滑动的步长, 默认为 `kernel_size`
- `Padding`: 边缘填充, 默认为 0
- `Dilation`: 空洞大小, 控制窗口内元素的间距, 用于扩大感受野
- `return_indices`, 是否返回最大值的索引, 默认为 `False`, 如果设为 `True`, 常用于配合 `MaxUnpool2d` 进行上采样恢复。
- `ceil_mode`: 尺寸计算方式, 默认为 `False` (向下取整); 设为 `True` 则向上取整, 确保即使除不尽也能保留边缘信息。

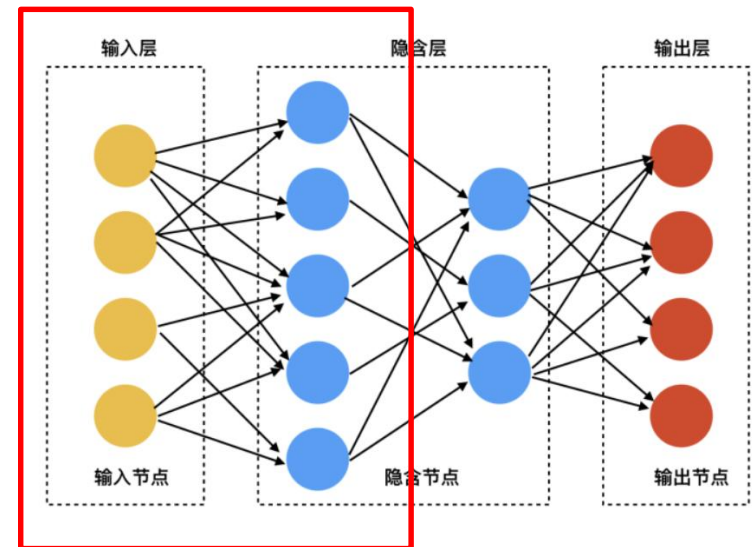
PyTorch: 全连接层

Linear <https://pytorch.org/docs/stable/generated/torch.nn.Linear.html#torch.nn.Linear>

```
CLASS torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None) [SOURCE]
```

- `in_features`: 输入特征维度
- `out_features`: 输出特征维度
- `bias`: 是否使用偏置项, 默认为 `True`; 如果后面紧接 `BatchNorm` 层, 通常设为 `False`。
- `device`: 指定设备, CPU还是CUDA
- `dtype`: 指定数据类型, `float32`还是`float16`

$$y = xW^T + b$$



➤ 层归一化与批归一化

LayerNorm

<https://docs.pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html>

```
class torch.nn.LayerNorm(normalized_shape, eps=1e-05, elementwise_affine=True, bias=True, device=None, dtype=None)
```

BatchNorm2d

<https://docs.pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html>

```
class torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True, device=None, dtype=None)
```

[\[source\]](#)

➤ 激活函数

ReLU

<https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html>

```
CLASS torch.nn.ReLU(inplace=False) \[SOURCE\]
```

LeNet网络模型

model.py



```
import torch.nn as nn

5 usages
class LeNet(nn.Module):
    def __init__(self):...

    def forward(self, x):...
```

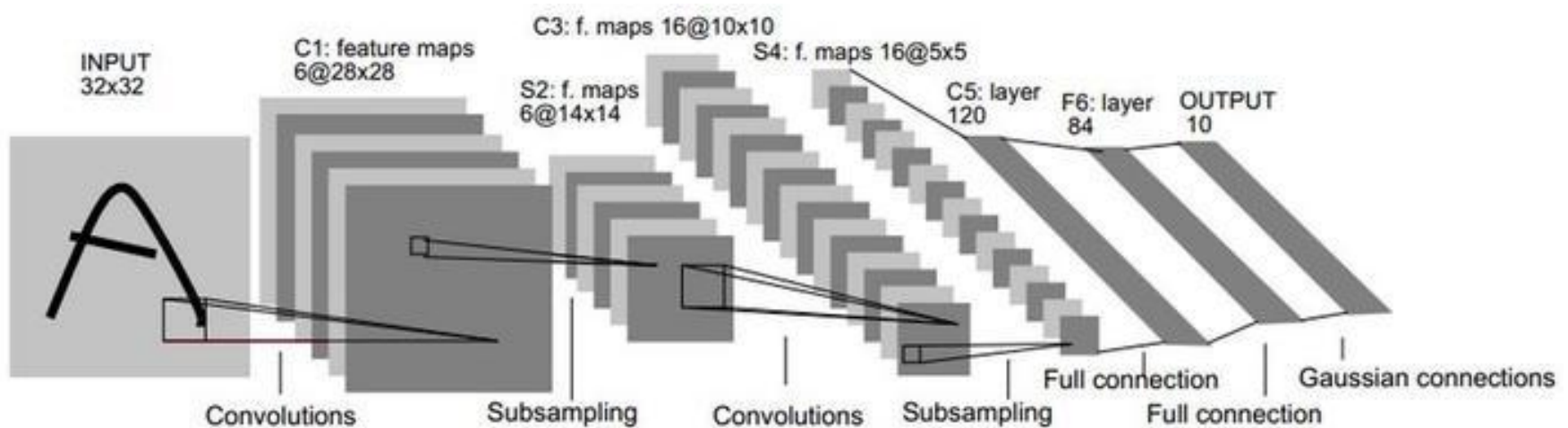
- 源码来源：
<https://github.com/WZMIAOMIAO/deep-learning-for-image-processing>
- 官方教程：
https://pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html

Learn the Basics || Quickstart || Tensors || Datasets & DataLoaders || Transforms || **Build Model** || Autograd || Optimization || Save & Load Model



LeNet网络模型

- 网络输入：32*32*3的彩色图像
- 网络输出：10类识别结果
- C1+S2：卷积核(5*5)*16，池化2*2
- C2+S2：卷积核(5*5)*32，池化2*2
- FC1：(5*5*32)*120
- FC2：120*84
- FC3：84*10





PyTorch的nn.Module 子类实现了__call__方法，让实例可以像函数一样被调用

特征提取

分类

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 5)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, 5)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
        self.relu = nn.ReLU()
```

```
def forward(self, x):
    x = self.relu(self.conv1(x)) # input(3, 32, 32) output(16, 28, 28)
    x = self.pool1(x)           # output(16, 14, 14)
    x = self.relu(self.conv2(x)) # output(32, 10, 10)
    x = self.pool2(x)           # output(32, 5, 5)
    x = x.view(-1, 32*5*5)      # output(32*5*5)
    x = self.relu(self.fc1(x))  # output(120)
    x = self.relu(self.fc2(x))  # output(84)
    x = self.fc3(x)             # output(10)
    return x
```

注意：输入x是四维，这里没显示batch维度

前向计算

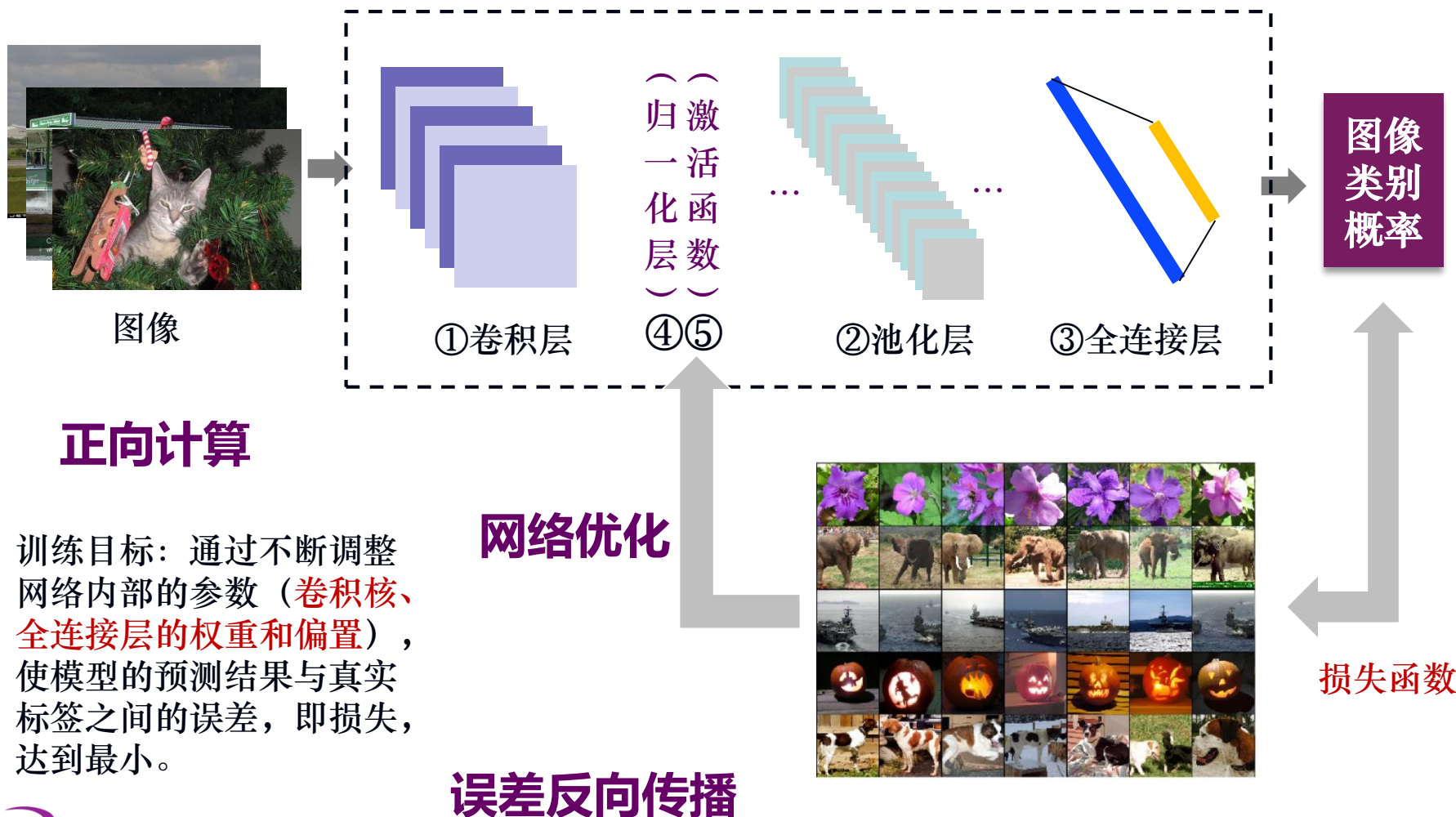
model.py

model.py

```
net = LeNet()
net.load_state_dict(torch.load('Lenet-1.pth')) # 加载模型
# 1. 用 PIL 读取图像 → NumPy 数组
img_big = Image.open('1.jpeg')
img = img_big.resize((32, 32))
img_np = np.array(img) # shape: (H, W, C), dtype: uint8
print(f"原始图像 shape: {img_np.shape}, dtype: {img_np.dtype}")
# 2. NumPy → Tensor (零拷贝)
img_tensor = torch.from_numpy(img_np)
print(f"Tensor shape: {img_tensor.shape}, dtype: {img_tensor.dtype}")
# 3. 调整维度: PyTorch 需要 (C, H, W)
img_tensor = img_tensor.permute(2, 0, 1) # (H,W,C) → (C,H,W)
print(f"转换后 shape: {img_tensor.shape}")
# 4. 归一化并转换为 float32
img_tensor = img_tensor.float() / 255.0
print(f"归一化后范围: [{img_tensor.min():.2f}, {img_tensor.max():.2f}]")
# 5. 添加 batch 维度
img_batch = img_tensor.unsqueeze(0) # (1, C, H, W)
print(f"Batch shape: {img_batch.shape}")
# 6. 模型推理
with torch.no_grad():
    outputs2 = net(img_batch)
# 7. Tensor → NumPy (用于可视化或保存)
output_np = outputs2.numpy()
print(output_np)
```

综合运用

6.2 网络训练与优化

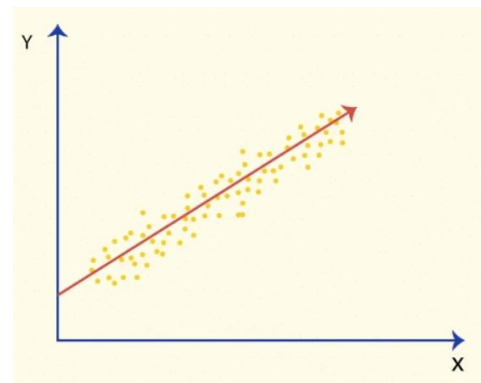


1) 损失函数

➤ 损失函数：计算网络输出与目标之间的误差

1. 均方误差(Mean Squared Error, MSE)

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$



2. 二元交叉熵(Binary Cross-Entropy, BCE)

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i)))$$

- x_i : 样本i
- y_i : 样本i的标签, 取值为{0,1}
- $p(x_i)$: 样本i预测为正样本的预测概率
- N : batch size

$$L = -\frac{1}{N} \sum_{i=1}^N (\log p_{i,y_i})$$

样本i在其所属类别的预测概率

1) 损失函数

- 损失函数：计算网络输出与目标之间的误差

3. 多分类交叉熵

$$L = -\frac{1}{N} \sum_{i=1}^N \left(\sum_j^C y_{i,j} \log p_{i,j} \right) = -\frac{1}{N} \sum_{i=1}^N (\log p_{i,y_i})$$

- C : 分类数
- $y_{i,j}$: 样本 i 在分类 j 的真实标签
- $p_{i,j}$: 样本 i 是分类 j 的预测概率

样本 i 在其所属
类别的预测概率

1) 损失函数

- 损失函数：计算网络输出与目标之间的误差

3. 多分类交叉熵

Softmax函数

$$L = -\frac{1}{N} \sum_{i=1}^N (\log p_{i,y_i}) \rightarrow p_{i,j} = \frac{e^{l_{i,y_i}}}{\sum_{k=1}^C e^{l_{i,k}}}$$

- C : 分类数
- $y_{i,j}$: 样本 i 在分类 j 的真实标签
- $p_{i,j}$: 样本 i 是分类 j 的预测概率

➤ Softmax交叉熵

$$L = -\frac{1}{N} \sum_{i=1}^N \left(\log \frac{e^{l_{i,y_i}}}{\sum_{k=1}^C e^{l_{i,k}}} \right)$$

2) 网络训练：数据集准备

- 数据集基本格式： $\{x_1, y_1\} \{x_2, y_2\}, \dots$
- 数据清洗、格式化与标准化
- 数据集划分，将完整的数据集划分为三个独立的子集：
 - 训练集 (Training Set): 用于模型学习，计算梯度并更新参数
 - 验证集 (Validation Set): 用于在训练过程中评估模型性能，调整超参数（如学习率），并监控是否过拟合
 - 测试集 (Test Set): 在模型训练完全结束后，用于最终评估模型的泛化能力

2) 网络训练：训练循环

➤ 批量算法和小批量算法

- 批量或确定性梯度算法：使用整个训练集
- 随机或者在线算法：每次只使用单个样本训练
- 小批量 (Mini-batch) 或小批量随机方法：每次采用一个以上样本但又不是全部

➤ 关键术语

- Epoch (轮次)：将整个训练集完整地遍历一次
- Batch Size (批次大小)：每次迭代中用于计算梯度的样本数量
- Iteration (迭代次数)：完成一个 Epoch 所需的批次数。计算公式为：总样本数 / Batch Size

2) 网络训练：训练循环

- **前向传播 (Forward Propagation)**: 将一个批次 (Mini-batch) 的训练数据输入网络，数据逐层传递，最终得到模型的预测输出。
- **计算损失 (Loss Calculation)**: 使用损失函数衡量模型的预测输出与真实标签之间的差距。损失值越大，说明预测越不准确。对于分类任务，常用交叉熵损失 (Cross-Entropy Loss)
- **反向传播 (Backward Propagation)**: 通过链式法则，从输出层开始，反向计算损失函数相对于网络中每一个参数（权重和偏置）的梯度，**梯度指明了参数应该如何调整才能减小损失**
- **参数更新 (Parameter Update)**: 使用优化器 (Optimizer) 根据**计算出的梯度**来更新网络的所有参数

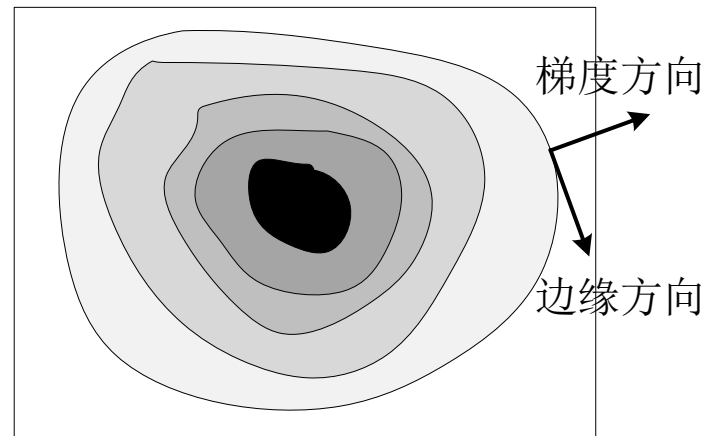
复习：图像中的梯度

- 边缘检测与滤波器
- 使用梯度(gradient)描述图像函数的变化，梯度方向是图像函数增长最大的方向
- 对于二维图像 $f(x, y)$ ，梯度定义为：

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T$$

- 梯度的幅值： $|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

- 梯度的方向： $\psi = \arg\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)$



2) 网络训练：优化器

- 优化器：如何根据梯度来更新模型参数（神经网络的权值和偏置），其目标是高效地找到损失函数的最小值

- 基本算法：随机梯度下降（SGD）

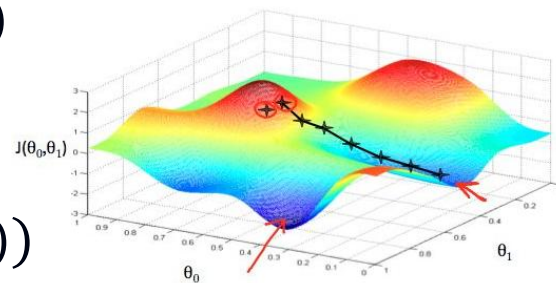
- 按照数据生成分布，随机抽取 m 个小批量样本
- 设要优化的模型参数为 θ ，计算损失函数 L 相对于 θ 的梯度均值，可以得到梯度的无偏估计，**这个梯度指出当前参数下，损失增长最快的方向**：

$$\frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)}))$$

- 沿着梯度的反方向更新参数，学习率为 ϵ

$$\theta \leftarrow \theta - \epsilon \frac{1}{m} \nabla_{\theta} (\sum_{i=1}^m L(f(x^{(i)}; \theta), y^{(i)}))$$

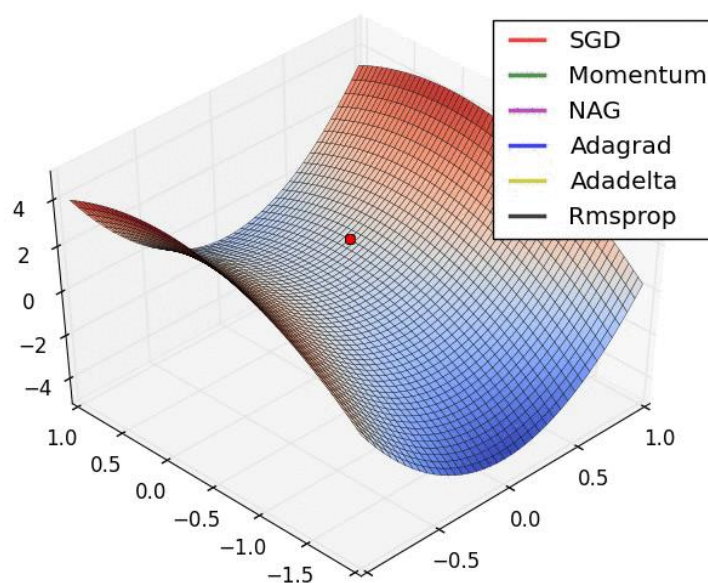
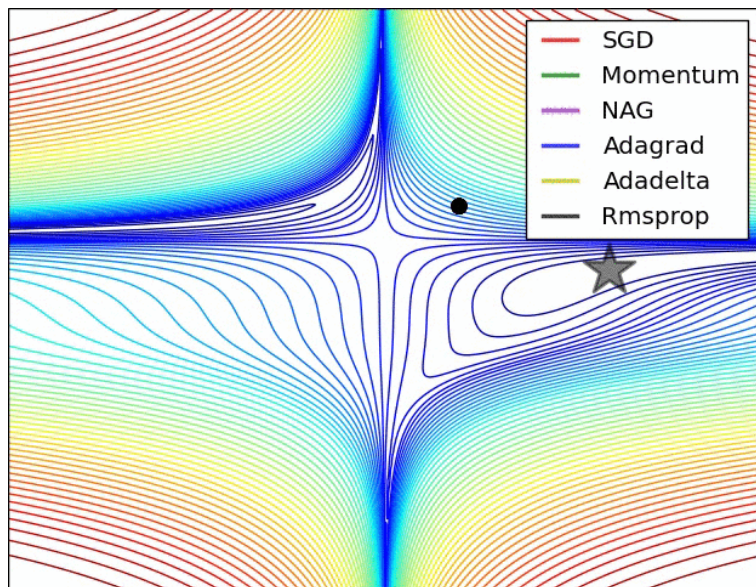
- 重复以上步骤，直到模型收敛（即损失不再显著下降）或达到预设的迭代次数



2) 网络训练：优化器

➤ SGD的改进版本

- 带动量的SGD (SGD with Momentum): 模拟物理中的动量概念，在更新参数时不仅考虑当前的梯度，还累积之前的更新方向
- 自适应学习率优化器：为模型中的每个参数自动调整学习率，使得训练过程更加稳定和高效。



2) 网络训练：优化器

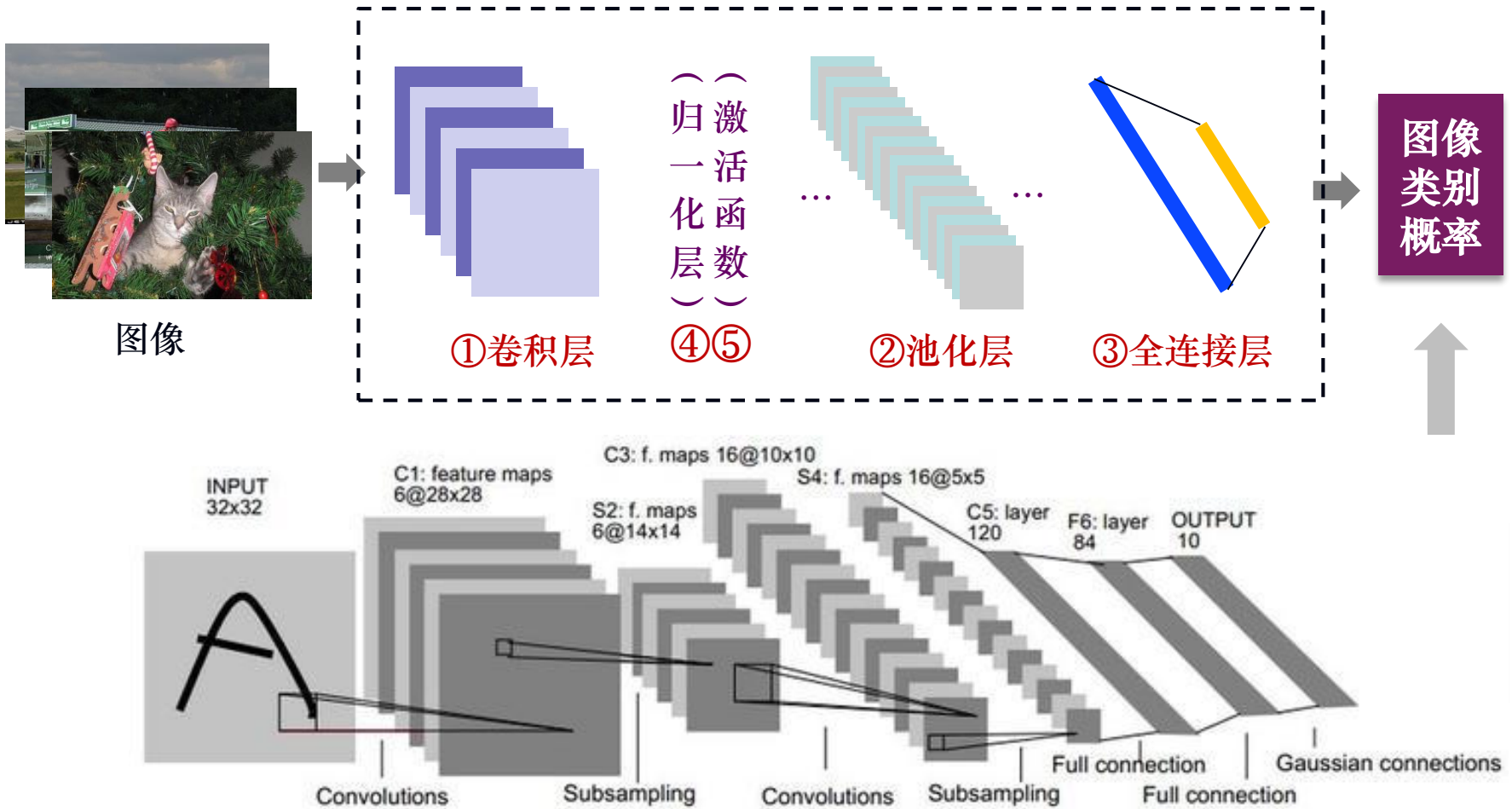
优化器	核心机制	优点	缺点
SGD	基础梯度更新	简单，可控	收敛慢，震荡，易陷局部最优
动量 Momentum	SGD + 惯性	加速收敛，抑制震荡	仍需手动调学习率
NAG动量	动量 + 前瞻	修正更及时，收敛更准	实现稍复杂
AdaGrad	自适应学习率+ 累积历史梯度平方	适合稀疏数据	学习率会过早归零
RMSProp	自适应学习率+ 仅保留近期梯度	解决AdaGrad停滞问题	需设置初始学习率
Adam	动量 + RMSProp	收敛快，自适应，鲁棒	极少数情况泛化略弱

2) 网络训练：优化器

➤ 选择优化建议

- 默认选择：如果不确定用什么，Adam 几乎总是最好的起点。它能节省大量调参时间，并提供不错的性能
- 追求极致性能：如果在做图像分类等任务，且发现Adam 训练出来的模型在测试集上效果不够好（泛化能力差），可以尝试切换到 SGD + Momentum，并配合学习率衰减策略进行精细调优
- 特殊数据：如果数据非常稀疏（例如推荐系统或自然语言处理中的长尾词），AdaGrad 或 RMSProp 可能会有奇效。
- 注意：没有哪个算法能脱颖而出，选择哪一个算法可以主要取决于使用者对算法的熟悉程度（以便调节超参数）

举例：LeNet训练



举例：LeNet训练

- 网络结构设计：`model.py`
- 数据集相关操作：`train.py`
 - 图像预处理：格式转换、标准化
 - 使用DataLoader将处理好的数据包装成一个迭代器
- 网络训练流程：`train.py`
 - 设定损失函数、优化器
 - 设定epoch、清空上一轮迭代遗留梯度
 - 训练循环：前向传播、计算损失、反向传播、参数更新
 - 训练验证：关闭梯度、模型预测、模型准确率计算
- 预测/推理：`predict.py`

数据集：CIFAR10



- 由60000张32*32彩色图像组成
- 图像有10个类，每个类有6000个图像
- 分别包含有50000个训练图像和10000个测试图像

airplane

automobile

bird

cat

deer

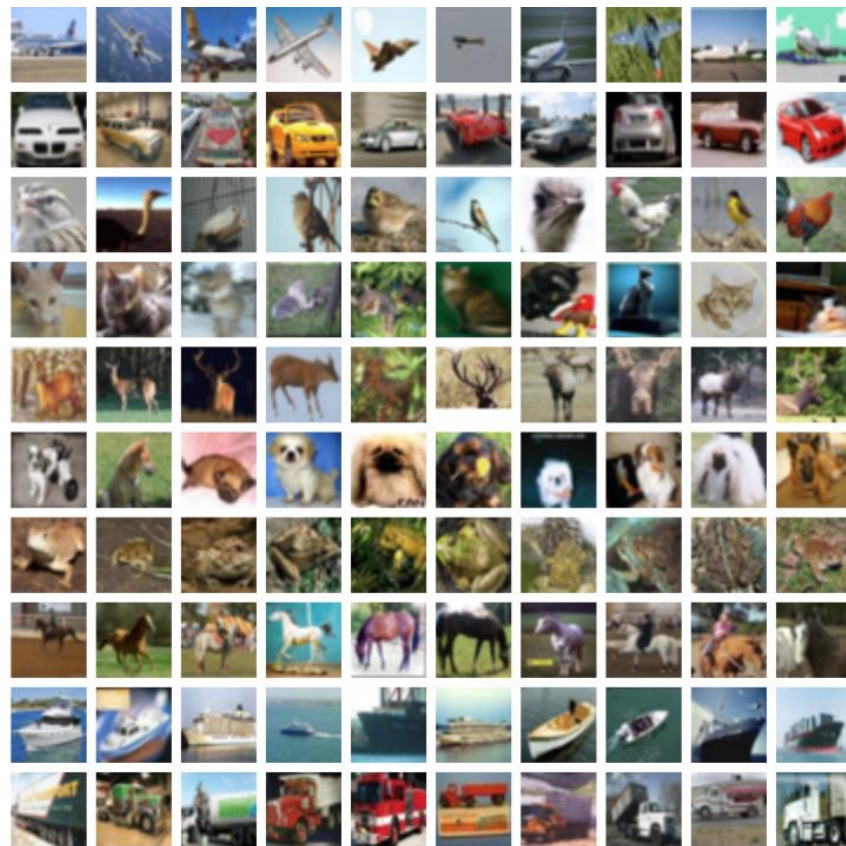
dog

frog

horse

ship

truck



数据准备

train.py

```
def main():
    transform = transforms.Compose(
        [transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    # 50000张训练图片
    # 第一次使用时要将download设置为True才会自动去下载数据集
    train_set = torchvision.datasets.CIFAR10(root='./data', train=True,
                                              download=False, transform=transform)
    train_loader = torch.utils.data.DataLoader(train_set, batch_size=32,
                                              shuffle=True, num_workers=0)

    # 10000张验证图片
    # 第一次使用时要将download设置为True才会自动去下载数据集
    val_set = torchvision.datasets.CIFAR10(root='./data', train=False,
                                             download=False, transform=transform)
    val_loader = torch.utils.data.DataLoader(val_set, batch_size=5000,
                                             shuffle=False, num_workers=0)

    val_data_iter = iter(val_loader)
    val_image, val_label = val_data_iter.next()
```

超参数

PyTorch: 图像预处理流水线



Compose

<https://docs.pytorch.org/vision/stable/generated/torchvision.transforms.Compose.html>

```
CLASS torchvision.transforms.Compose(transforms) [SOURCE]
```

在深度学习中，原始图片在送入神经网络前，需要经过一系列处理，包括调整大小、裁剪、转换成张量、归一化等

`transforms.Compose` 的作用是把些零散的图像变换操作按顺序组合起来，形成一个单一的、可复用的处理流程

- `transforms.RandomResizedCrop`: 将图片先按随机的大小和长宽比进行裁剪，然后再缩放到指定尺寸
- `transforms.RandomHorizontalFlip`: 以一定概率随机水平翻转图像
- `transforms.ToTensor`: 将原始图像转换为Tensor格式
- `transforms.Normalize(mean, std)`: 对图像进行归一化处理
- `transforms.Resize`: 调整图像大小，通常用在预测时
-

PyTorch: 图像预处理流水线

```
transforms = v2.Compose([
    v2.RandomResizedCrop(size=(224, 224), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
out = transforms(img)
plot([img, out])
```

https://docs.pytorch.org/vision/stable/auto_examples/transforms/plot_transforms_getting_started.html



PyTorch: 数据加载

<https://docs.pytorch.org/docs/2.11/data.html#torch.utils.data.DataLoader>

```
DataLoader(dataset, batch_size=1, shuffle=False, sampler=None,
            batch_sampler=None, num_workers=0, collate_fn=None,
            pin_memory=False, drop_last=False, timeout=0,
            worker_init_fn=None, *, prefetch_factor=2,
            persistent_workers=False)
```

在深度学习训练中，模型无法一次性吞下所有数据，DataLoader将数据集包装成一个可迭代的对象，为模型训练提供批量（batch）、打乱（shuffle）、多进程等数据服务

- dataset: 要加载的数据集，必须是Dataset的子类实例
- batch_size: 批大小，控制每次迭代返回的样本数量
- shuffle: 是否在每个 epoch 打乱数据顺序，训练集设为 True，测试集设为 False
- num_workers: 用于数据加载的子进程数量，Linux/Mac可设2-8，Windows常设为0

网络训练

```
net = LeNet()  
loss_function = nn.CrossEntropyLoss()  
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

```
for epoch in range(5): # loop over the dataset multiple times
```

```
    running_loss = 0.0
```

```
    for step, data in enumerate(train_loader, start=0):
```

```
        # get the inputs; data is a list of [inputs, labels]
```

```
        inputs, labels = data
```

```
        # zero the parameter gradients
```

```
        optimizer.zero_grad() → 清空上一轮迭代遗留的梯度
```

```
        # forward + backward + optimize
```

```
        ① outputs = net(inputs)
```

```
        ② loss = loss_function(outputs, labels)
```

```
        ③ loss.backward()
```

```
        ④ optimizer.step()
```

自动计算 loss 相对于网络中每一个参数的梯度，梯度被计算出来并存放在每个参数的 .grad 属性中

超参数

训练验证

train.py



PyTorch 中的一个上下文管理器，在这个环境内的所有计算都不会记录梯度

```
# print statistics
running_loss += loss.item()
if step % 500 == 499:    # print every 500 mini-batches
    with torch.no_grad():
        outputs = net(val_image) # [batch, 10]
        predict_y = torch.max(outputs, dim=1)[1]
        accuracy = torch.eq(predict_y, val_label).sum().item() / val_label.size(0)

    print('[%d, %5d] train_loss: %.3f  test_accuracy: %.3f' %
          (epoch + 1, step + 1, running_loss / 500, accuracy))
    running_loss = 0.0

print('Finished Training')

save_path = './Lenet.pth'
torch.save(net.state_dict(), save_path)
```

```
[1,  500] train_loss: 1.735  test_accuracy: 0.443
[1, 1000] train_loss: 1.431  test_accuracy: 0.506
[2,  500] train_loss: 1.211  test_accuracy: 0.569
[2, 1000] train_loss: 1.158  test_accuracy: 0.593
[3,  500] train_loss: 1.016  test_accuracy: 0.632
[3, 1000] train_loss: 1.010  test_accuracy: 0.635
[4,  500] train_loss: 0.903  test_accuracy: 0.650
[4, 1000] train_loss: 0.918  test_accuracy: 0.659
[5,  500] train_loss: 0.820  test_accuracy: 0.657
[5, 1000] train_loss: 0.837  test_accuracy: 0.686
```



预测/推理

predict.py

```
def main():
    transform = transforms.Compose(
        [transforms.Resize((32, 32)),
         transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

    classes = ('plane', 'car', 'bird', 'cat',
               'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

    net = LeNet()
    net.load_state_dict(torch.load('Lenet.pth'))

    im = Image.open('1.jpg')
    im = transform(im) # [C, H, W]
    im = torch.unsqueeze(im, dim=0) # [N, C, H, W]

    with torch.no_grad():
        outputs = net(im)
        predict = torch.max(outputs, dim=1)[1].numpy()
    print(classes[int(predict)])
```


➤ 第六章 卷积神经网络

- 6.1 卷积神经网络的组成：卷积层、池化层、全连接层、归一化层、激活函数
- 补充1：Python基础：
 - 基本语法、内置数据结构、numpy多维数组
- 补充2：PyTorch基础
 - Tensor、PyTorch的卷积神经网络接口
 - LeNet网络模型
- 6.2 网络训练与优化
 - 损失函数、网络训练（数据集、训练循环、优化器）
 - LeNet训练

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University